

The toral subgroup of $G(\mathfrak{m})$ commutes with the unipotent subgroup up to a unipotent factor: a formal verification

Abstract

For the group $G(\mathfrak{m})$, defined by generators and relations attached to the Monster Lie algebra, let H be the toral subgroup and U the unipotent subgroup. Lemma 3.5 of the source states that for all $h \in H$ and $u \in U$ there is a $u' \in U$ with $uh = hu'$. We present a Lean 4/Mathlib formalization of this lemma, where the coefficients $c(j)$ of the modular function J are replaced by an arbitrary function $c : \mathbb{N} \rightarrow \mathbb{N}$. The proof uses the previously verified formal version of Theorem 3.4, that H normalizes U , together with one elementary group identity.

Source	Carbone et al., G-Simple (draft), GSimple_Lemma_3_5
Library	libraries/gsimple.lean, 58 declarations
Lean	leanprover/lean4:v4.23.0, Mathlib 37df177aaa
Machine checks	compiles; no sorry; statement matches target; module build
Axioms	propext, Classical.choice, Quot.sound
Independent review	FAITHFUL (claude-opus-5-5)
Agents	captain claude-opus-5-5; solvers aiprover; report claude-opus-5-5
Run	srv_20261006_173043_GSimple_Lemma_3_5

Contents

1 Statement	1
2 Formalization	3
3 Proof	8
3.1 Overview	8
3.2 Proof of the theorem	9
4 Independent review	9
A Complete Lean file	11

1 Statement

Let $\mathbb{C}$ be the complex numbers and $\mathbb{N}$ the natural numbers. Let $c(j)$ be the coefficient of q^j in the modular function

$$J(q) = \sum_{j \geq -1} c(j)q^j = \frac{1}{q} + 196884q + 21493760q^2 + 864299970q^3 + \dots,$$

so that $c(-1) = 1$, $c(0) = 0$ and $c(1) = 196884$. Define the index set

$$E = \{(\ell, j, k) \mid j \in \mathbb{N}, 1 \leq k \leq c(j), 0 \leq \ell < j\}$$

and, for $0 \leq \ell < j$, the number

$$c_{\ell j} := (-1)^{\ell+1} \binom{j-1}{\ell} (\ell+1)(j-\ell).$$

Let $\mathcal{X}$ be the set of symbols

$$\mathcal{X} = \{H_1(s), H_2(s), X_{-1}(u), Y_{-1}(u), X_{\ell,jk}(u), Y_{\ell,jk}(u) \mid s \in \mathbb{C}^\times, u \in \mathbb{C}, (\ell, j, k) \in E\},$$

and let $F(\mathcal{X})$ be the free group on $\mathcal{X}$. The symbols $X_{-1}(u)$, $Y_{-1}(u)$, $X_{\ell,jk}(u)$ and $Y_{\ell,jk}(u)$ are called the *unipotent generators*. The commutator is $(a, b) = aba^{-1}b^{-1}$.

The set $\mathcal{R} \subseteq F(\mathcal{X})$ consists of the following relations, for all $s, t \in \mathbb{C}^\times$, $u, v \in \mathbb{C}$ and $(\ell, j, k), (m, p, q) \in E$. The auxiliary elements are

$$\tilde{w}_{-1}(s) := X_{-1}(s) Y_{-1}(-s^{-1}) X_{-1}(s), \quad \tilde{w}_{-1} := \tilde{w}_{-1}(1), \quad (\text{Re:0})$$

$$\tilde{w}_{\ell,jk}(s) := X_{\ell,jk}(s) Y_{\ell,jk}\left(\frac{-s^{-1}}{c_{\ell j}}\right) X_{\ell,jk}(s), \quad \tilde{w}_{\ell,jk} := \tilde{w}_{\ell,jk}(1). \quad (\text{Im:0})$$

The relations are:

$$H_1(s)H_1(t) = H_1(st), \quad (\text{H:1a})$$

$$H_2(s)H_2(t) = H_2(st), \quad (\text{H:1b})$$

$$H_1(s)H_2(t) = H_2(t)H_1(s), \quad (\text{H:2})$$

$$X_{-1}(u)X_{-1}(v) = X_{-1}(u+v), \quad (\text{Re:1a})$$

$$Y_{-1}(u)Y_{-1}(v) = Y_{-1}(u+v), \quad (\text{Re:1b})$$

$$Y_{-1}(-t)X_{-1}(s)Y_{-1}(t) = X_{-1}(-t^{-1})Y_{-1}(-t^2s)X_{-1}(t^{-1}), \quad (\text{Re:2})$$

$$\tilde{w}_{-1}(s)\tilde{w}_{-1} = H_1(-s)H_2(-s^{-1}), \quad (\text{Re:3})$$

$$\tilde{w}_{-1}X_{-1}(u)\tilde{w}_{-1}^{-1} = Y_{-1}(-u), \quad (\text{Re:4a})$$

$$\tilde{w}_{-1}Y_{-1}(u)\tilde{w}_{-1}^{-1} = X_{-1}(-u), \quad (\text{Re:4b})$$

$$\tilde{w}_{-1}H_1(s)\tilde{w}_{-1}^{-1} = H_2(s), \quad (\text{Re:5a})$$

$$\tilde{w}_{-1}H_2(s)\tilde{w}_{-1}^{-1} = H_1(s), \quad (\text{Re:5b})$$

$$H_1(s)X_{-1}(u)H_1(s)^{-1} = X_{-1}(su), \quad (\text{Re:6a})$$

$$H_2(s)X_{-1}(u)H_2(s)^{-1} = X_{-1}(s^{-1}u), \quad (\text{Re:6b})$$

$$H_1(s)Y_{-1}(u)H_1(s)^{-1} = Y_{-1}(s^{-1}u), \quad (\text{Re:6c})$$

$$H_2(s)Y_{-1}(u)H_2(s)^{-1} = Y_{-1}(su), \quad (\text{Re:6d})$$

$$X_{\ell,jk}(u)X_{\ell,jk}(v) = X_{\ell,jk}(u+v), \quad (\text{Im:1a})$$

$$Y_{\ell,jk}(u)Y_{\ell,jk}(v) = Y_{\ell,jk}(u+v), \quad (\text{Im:1b})$$

$$Y_{\ell,jk}(-t)X_{\ell,jk}(s)Y_{\ell,jk}(t) = X_{\ell,jk}\left(\frac{-t^{-1}}{c_{\ell j}}\right)Y_{\ell,jk}(-c_{\ell j}t^2s)X_{\ell,jk}\left(\frac{t^{-1}}{c_{\ell j}}\right), \quad (\text{Im:2})$$

$$\tilde{w}_{\ell,jk}(s^{(\ell+1)(j-\ell)})\tilde{w}_{\ell,jk} = H_1((-s)^{j-\ell})H_2((-s)^{\ell+1}), \quad (\text{Im:3})$$

$$\tilde{w}_{\ell,jk} X_{\ell,jk}(u) \tilde{w}_{\ell,jk}^{-1} = Y_{\ell,jk}\left(\frac{-u}{c_{\ell j}}\right), \quad (\text{Im:4a})$$

$$\tilde{w}_{\ell,jk} Y_{\ell,jk}(u) \tilde{w}_{\ell,jk}^{-1} = X_{\ell,jk}(-c_{\ell j} u), \quad (\text{Im:4b})$$

$$\tilde{w}_{\ell,jk} H_1(s^{j-\ell}) \tilde{w}_{\ell,jk}^{-1} = H_2(s^{-(\ell+1)}), \quad (\text{Im:5a})$$

$$\tilde{w}_{\ell,jk} H_2(s^{\ell+1}) \tilde{w}_{\ell,jk}^{-1} = H_1(s^{-(j-\ell)}), \quad (\text{Im:5b})$$

$$H_1(s) X_{\ell,jk}(u) H_1(s)^{-1} = X_{\ell,jk}(s^{\ell+1} u), \quad (\text{Im:6a})$$

$$H_2(s) X_{\ell,jk}(u) H_2(s)^{-1} = X_{\ell,jk}(s^{j-\ell} u), \quad (\text{Im:6b})$$

$$H_1(s) Y_{\ell,jk}(u) H_1(s)^{-1} = Y_{\ell,jk}(s^{-(\ell+1)} u), \quad (\text{Im:6c})$$

$$H_2(s) Y_{\ell,jk}(u) H_2(s)^{-1} = Y_{\ell,jk}(s^{-(j-\ell)} u), \quad (\text{Im:6d})$$

$$(X_{-1}(u), X_{j-1,jk}(v)) = 1, \quad (\text{U:1a})$$

$$(Y_{-1}(u), Y_{j-1,jk}(v)) = 1, \quad (\text{U:1b})$$

$$(Y_{-1}(u), X_{0,jk}(v)) = 1, \quad (\text{U:1c})$$

$$(X_{-1}(u), Y_{0,jk}(v)) = 1, \quad (\text{U:1d})$$

$$(X_{\ell,jk}(u), Y_{m,pq}(v)) = 1 \quad \text{if } j \neq p \text{ or } k \neq q \text{ or } |\ell - m| > 1, \quad (\text{U:2})$$

$$(X_{1,2k}(u), Y_{0,2k}(v)) = X_{-1}(uv), \quad (\text{U:3a})$$

$$(X_{0,2k}(u), Y_{1,2k}(v)) = Y_{-1}(uv), \quad (\text{U:3b})$$

$$\tilde{w}_{-1} X_{\ell,jk}(u) \tilde{w}_{-1}^{-1} = X_{j-1-\ell,jk}((-1)^{j-\ell-1} u), \quad (\text{U:4a})$$

$$\tilde{w}_{-1} Y_{\ell,jk}(u) \tilde{w}_{-1}^{-1} = Y_{j-1-\ell,jk}((-1)^{j-\ell-1} u). \quad (\text{U:4b})$$

The group is $G(\mathfrak{m}) = \langle \mathcal{X} \mid \mathcal{R} \rangle = F(\mathcal{X})/N_{\mathcal{R}}$, where $N_{\mathcal{R}}$ is the normal closure of $\mathcal{R}$.

The *toral subgroup* H is the subgroup of $G(\mathfrak{m})$ generated by $\{H_1(s), H_2(s) \mid s \in \mathbb{C}^\times\}$. The *unipotent subgroup* U is the subgroup of $G(\mathfrak{m})$ generated by all unipotent generators $X_{-1}(u)$, $Y_{-1}(u)$, $X_{\ell,jk}(u)$, $Y_{\ell,jk}(u)$, with $u \in \mathbb{C}$ and $(\ell, j, k) \in E$.

Theorem 1.1 (Lemma 3.5 of the source). *For any $h \in H$ and $u \in U$ there exists an element $u' \in U$ such that $uh = hu'$.*

The source proof cites Theorem 3.4, which states that H normalizes U , that is, $huh^{-1} \in U$ for all $h \in H$ and $u \in U$. It sets $u' = h^{-1}uh$ and multiplies on the left by h .

2 Formalization

The formalization builds on a library of previously verified declarations. These come from the earlier formalizations of Theorems 3.3 and 3.4 of the source. The library defines $G(\mathfrak{m})$ as a presented group in Mathlib, together with the subgroups H and U . All library declarations live in the namespace $\mathbf{GM}$.

The coefficients. Throughout, the coefficients $c(j)$ of J are replaced by an arbitrary parameter $c : \mathbb{N} \rightarrow \mathbb{N}$. The proof uses c only through the index set. The theorem verified here carries no hypothesis on c ; in particular it does not need $c(2) \geq 1$. It therefore holds for the actual coefficients of J .

Index set. For $\ell, j, k \in \mathbb{N}$, $\text{Valid}_c(\ell, j, k)$ is the proposition $\ell < j \wedge 1 \leq k \wedge k \leq c(j)$. Thus $(\ell, j, k) \in E$ if and only if $\text{Valid}_c(\ell, j, k)$ holds when c is the coefficient function of J . The case $j = 0$ is excluded in both settings: in the source because $c(0) = 0$, and in the formalization because $\ell < j$.

Lean: Valid — library, verified earlier

```

| /-- Index set E:  $(\ell, j, k)$  with  $0 \leq \ell < j$  and  $1 \leq k \leq c(j)$ . -/
| def Valid (c :  $\mathbb{N} \rightarrow \mathbb{N}$ ) (l j k :  $\mathbb{N}$ ) : Prop := 1 < j  $\wedge$  1  $\leq$  k  $\wedge$  k  $\leq$  c j

```

Generators. The type Gen_c has six kinds of elements:

- $H_1(s)$ and $H_2(s)$ for $s \in \mathbb{C}^\times$;
- $X_{-1}(u)$ and $Y_{-1}(u)$ for $u \in \mathbb{C}$;
- $X_{\ell,jk}(u)$ and $Y_{\ell,jk}(u)$ for $\ell, j, k \in \mathbb{N}$, a proof of $\text{Valid}_c(\ell, j, k)$, and $u \in \mathbb{C}$.

Because proofs of a proposition are all equal, the last two kinds are indexed exactly by $E \times \mathbb{C}$. Hence Gen_c is the set $\mathcal{X}$, and F_c is the free group $F(\mathcal{X})$.

Lean: Gen — library, verified earlier

```

| /-- The generating symbols. -/
| inductive Gen (c :  $\mathbb{N} \rightarrow \mathbb{N}$ ) : Type
| H1 :  $\mathbb{C}^\times \rightarrow \text{Gen } c$ 
| H2 :  $\mathbb{C}^\times \rightarrow \text{Gen } c$ 
| Xm :  $\mathbb{C} \rightarrow \text{Gen } c$ 
| Ym :  $\mathbb{C} \rightarrow \text{Gen } c$ 
| X (l j k :  $\mathbb{N}$ ) (h : Valid c l j k) (u :  $\mathbb{C}$ ) : Gen c
| Y (l j k :  $\mathbb{N}$ ) (h : Valid c l j k) (u :  $\mathbb{C}$ ) : Gen c

```

Lean: F — library, verified earlier

```

| abbrev F (c :  $\mathbb{N} \rightarrow \mathbb{N}$ ) := FreeGroup (Gen c)

```

The maps h_1, h_2, x_m, y_m, x, y send a parameter to the corresponding generator of F_c . Their images are $H_1(s)$, $H_2(s)$, $X_{-1}(u)$, $Y_{-1}(u)$, $X_{\ell,jk}(u)$ and $Y_{\ell,jk}(u)$ respectively.

Lean: h1 — library, verified earlier

```

| def h1 {c :  $\mathbb{N} \rightarrow \mathbb{N}$ } (s :  $\mathbb{C}^\times$ ) : F c := FreeGroup.of (Gen.H1 s)

```

Lean: h2 — library, verified earlier

```

| def h2 {c :  $\mathbb{N} \rightarrow \mathbb{N}$ } (s :  $\mathbb{C}^\times$ ) : F c := FreeGroup.of (Gen.H2 s)

```

Lean: xm — library, verified earlier

```

| def xm {c :  $\mathbb{N} \rightarrow \mathbb{N}$ } (u :  $\mathbb{C}$ ) : F c := FreeGroup.of (Gen.Xm u)

```

Lean: ym — library, verified earlier

```
| def ym {c : ℕ → ℕ} (u : ℂ) : F c := FreeGroup.of (Gen.Ym u)
```

Lean: xx — library, verified earlier

```
| def xx {c : ℕ → ℕ} (l j k : ℕ) (h : Valid c l j k) (u : ℂ) : F c :=
  FreeGroup.of (Gen.X l j k h u)
```

Lean: yy — library, verified earlier

```
| def yy {c : ℕ → ℕ} (l j k : ℕ) (h : Valid c l j k) (u : ℂ) : F c :=
  FreeGroup.of (Gen.Y l j k h u)
```

Constants and auxiliary words. The complex number $cc(\ell, j)$ is

$$cc(\ell, j) = (-1)^{\ell+1} \binom{j \dot{-} 1}{\ell} (\ell + 1)(j \dot{-} \ell),$$

where $\dot{-}$ is truncated subtraction on $\mathbb{N}$. When $\ell < j$ this equals $c_{\ell j}$, and it is then nonzero. As a result, the divisions by $c_{\ell j}$ in the relations below are genuine divisions, even though Lean sets $x/0 = 0$.

Lean: cc — library, verified earlier

```
| /-- c_{\ell j} = (-1)^{\ell+1} * binom(j-1, \ell) * (\ell+1) * (j-\ell). -/
| noncomputable def cc (l j : ℕ) : ℂ :=
  (-1 : ℂ) ^ (l + 1) * ((j - 1).choose l : ℕ) : ℂ * ((l : ℂ) + 1) * ((j - l : ℕ) : ℂ)
```

The commutator is $cm(a, b) = aba^{-1}b^{-1}$, and the relator encoding the equation $a = b$ is $eqr(a, b) = ab^{-1}$.

Lean: cm — library, verified earlier

```
| /-- commutator (a,b) = a b a^{-1} b^{-1} -/
| def cm {c : ℕ → ℕ} (a b : F c) : F c := a * b * a^{-1} * b^{-1}
```

Lean: eqr — library, verified earlier

```
| /-- relator for the equation a = b -/
| def eqr {c : ℕ → ℕ} (a b : F c) : F c := a * b^{-1}
```

The words $wm(s)$ and $ww_{\ell, jk}(s)$, for $s \in \mathbb{C}^\times$, are the words $\tilde{w}_{-1}(s)$ of (Re:0) and $\tilde{w}_{\ell, jk}(s)$ of (Im:0).

Lean: wm — library, verified earlier

```
noncomputable def wm {c : ℕ → ℕ} (s : ℂ×) : F c :=
  xm (s : ℂ) * ym (-(s : ℂ)-1) * xm (s : ℂ)
```

Lean: ww — library, verified earlier

```
noncomputable def ww {c : ℕ → ℕ} (l j k : ℕ) (h : Valid c l j k) (s : ℂ×) : F c :=
  xx l j k h (s : ℂ) * yy l j k h (-(s : ℂ)-1) / cc l j * xx l j k h (s : ℂ)
```

Relations. The predicate Rel_c on F_c has one constructor per tagged relation. The constructors H1a, H1b, H2, Re1a, ..., U4b correspond to (H:1a), (H:1b), (H:2), (Re:1a), ..., (U:4b). An equation $a = b$ is recorded as the relator ab^{-1} , and a relation $(a, b) = 1$ as the relator $aba^{-1}b^{-1}$. The parameters range as in the source: $s, t \in \mathbb{C}^\times$, $u, v \in \mathbb{C}$, and index triples carry a validity proof.

Four points of encoding should be noted.

- In (U:1a) and (U:1b) the triple is $(j - 1, j, k)$, and in (U:1c) and (U:1d) it is $(0, j, k)$.
- In (U:2) the side condition is $j \neq p \vee k \neq q \vee |\ell - m| > 1$, with $|\ell - m|$ computed in $\mathbb{Z}$.
- In (U:3a) and (U:3b) both triples $(1, 2, k)$ and $(0, 2, k)$ must be valid.
- In (U:4a) and (U:4b) the target triple $(j - 1 - \ell, j, k)$ is also required to be valid. This is automatic when $(\ell, j, k) \in E$. All truncated subtractions in exponents and indices are true differences whenever $\ell < j$.

Powers such as $s^{(\ell+1)(j-\ell)}$ and $(-s)^{j-\ell}$ in (Im:3), and inverses in (Im:5a) and (Im:5b), are taken in $\mathbb{C}^\times$. For example, $H_2(s^{-(\ell+1)})$ is written as $h_2((s^{\ell+1})^{-1})$.

Lean: Rel — library, verified earlier

```
/-- The defining relators. -/
inductive Rel (c : ℕ → ℕ) : F c → Prop
| H1a (s t : ℂ×) : Rel c (eqv (h1 s * h1 t) (h1 (s * t)))
| H1b (s t : ℂ×) : Rel c (eqv (h2 s * h2 t) (h2 (s * t)))
| H2 (s t : ℂ×) : Rel c (eqv (h1 s * h2 t) (h2 t * h1 s))
| Re1a (u v : ℂ) : Rel c (eqv (xm u * xm v) (xm (u + v)))
| Re1b (u v : ℂ) : Rel c (eqv (ym u * ym v) (ym (u + v)))
| Re2 (s t : ℂ×) : Rel c (eqv (ym (-(t : ℂ)) * xm (s : ℂ) * ym (t : ℂ))
  (xm (-(t : ℂ)-1) * ym (-(t : ℂ) ^ 2 * (s : ℂ)) * xm ((t : ℂ)-1)))
| Re3 (s : ℂ×) : Rel c (eqv (wm s * wm 1) (h1 (-s) * h2 (-(s-1))))
| Re4a (u : ℂ) : Rel c (eqv (wm 1 * xm u * (wm 1)-1) (ym (-u)))
| Re4b (u : ℂ) : Rel c (eqv (wm 1 * ym u * (wm 1)-1) (xm (-u)))
| Re5a (s : ℂ×) : Rel c (eqv (wm 1 * h1 s * (wm 1)-1) (h2 s))
| Re5b (s : ℂ×) : Rel c (eqv (wm 1 * h2 s * (wm 1)-1) (h1 s))
| Re6a (s : ℂ×) (u : ℂ) : Rel c (eqv (h1 s * xm u * (h1 s)-1) (xm ((s : ℂ) * u)))
| Re6b (s : ℂ×) (u : ℂ) : Rel c (eqv (h2 s * xm u * (h2 s)-1) (xm ((s : ℂ)-1 * u)))
| Re6c (s : ℂ×) (u : ℂ) : Rel c (eqv (h1 s * ym u * (h1 s)-1) (ym ((s : ℂ)-1 * u)))
| Re6d (s : ℂ×) (u : ℂ) : Rel c (eqv (h2 s * ym u * (h2 s)-1) (ym ((s : ℂ) * u)))
| Im1a (l j k : ℕ) (h : Valid c l j k) (u v : ℂ) :
  Rel c (eqv (xx l j k h u * xx l j k h v) (xx l j k h (u + v)))
| Im1b (l j k : ℕ) (h : Valid c l j k) (u v : ℂ) :
  Rel c (eqv (yy l j k h u * yy l j k h v) (yy l j k h (u + v)))
| Im2 (l j k : ℕ) (h : Valid c l j k) (s t : ℂ×) :
```

```

Rel c (eqr (yy 1 j k h (-(t : ℂ)) * xx 1 j k h (s : ℂ) * yy 1 j k h (t : ℂ))
  (xx 1 j k h (-(t : ℂ)-1 / cc 1 j) *
    yy 1 j k h (-(cc 1 j) * (t : ℂ) ^ 2 * (s : ℂ)) *
    xx 1 j k h ((t : ℂ)-1 / cc 1 j)))
| Im3 (1 j k : ℕ) (h : Valid c 1 j k) (s : ℂ×) :
  Rel c (eqr (ww 1 j k h (s ^ ((1 + 1) * (j - 1))) * ww 1 j k h 1)
    (h1 ((-s) ^ (j - 1)) * h2 ((-s) ^ (1 + 1))))
| Im4a (1 j k : ℕ) (h : Valid c 1 j k) (u : ℂ) :
  Rel c (eqr (ww 1 j k h 1 * xx 1 j k h u * (ww 1 j k h 1)-1) (yy 1 j k h (-u / cc 1 j)))
| Im4b (1 j k : ℕ) (h : Valid c 1 j k) (u : ℂ) :
  Rel c (eqr (ww 1 j k h 1 * yy 1 j k h u * (ww 1 j k h 1)-1) (xx 1 j k h (-(cc 1 j) * u)))
| Im5a (1 j k : ℕ) (h : Valid c 1 j k) (s : ℂ×) :
  Rel c (eqr (ww 1 j k h 1 * h1 (s ^ (j - 1)) * (ww 1 j k h 1)-1) (h2 ((s ^ (1 + 1))-1)))
| Im5b (1 j k : ℕ) (h : Valid c 1 j k) (s : ℂ×) :
  Rel c (eqr (ww 1 j k h 1 * h2 (s ^ (1 + 1)) * (ww 1 j k h 1)-1) (h1 ((s ^ (j - 1))-1)))
| Im6a (1 j k : ℕ) (h : Valid c 1 j k) (s : ℂ×) (u : ℂ) :
  Rel c (eqr (h1 s * xx 1 j k h u * (h1 s)-1) (xx 1 j k h ((s : ℂ) ^ (1 + 1) * u)))
| Im6b (1 j k : ℕ) (h : Valid c 1 j k) (s : ℂ×) (u : ℂ) :
  Rel c (eqr (h2 s * xx 1 j k h u * (h2 s)-1) (xx 1 j k h ((s : ℂ) ^ (j - 1) * u)))
| Im6c (1 j k : ℕ) (h : Valid c 1 j k) (s : ℂ×) (u : ℂ) :
  Rel c (eqr (h1 s * yy 1 j k h u * (h1 s)-1) (yy 1 j k h (((s : ℂ) ^ (1 + 1))-1 * u)))
| Im6d (1 j k : ℕ) (h : Valid c 1 j k) (s : ℂ×) (u : ℂ) :
  Rel c (eqr (h2 s * yy 1 j k h u * (h2 s)-1) (yy 1 j k h (((s : ℂ) ^ (j - 1))-1 * u)))
| U1a (j k : ℕ) (h : Valid c (j - 1) j k) (u v : ℂ) :
  Rel c (cm (xm u) (xx (j - 1) j k h v))
| U1b (j k : ℕ) (h : Valid c (j - 1) j k) (u v : ℂ) :
  Rel c (cm (ym u) (yy (j - 1) j k h v))
| U1c (j k : ℕ) (h : Valid c 0 j k) (u v : ℂ) :
  Rel c (cm (ym u) (xx 0 j k h v))
| U1d (j k : ℕ) (h : Valid c 0 j k) (u v : ℂ) :
  Rel c (cm (xm u) (yy 0 j k h v))
| U2 (1 j k : ℕ) (h : Valid c 1 j k) (m p q : ℕ) (h' : Valid c m p q) (u v : ℂ)
  (hne : j ≠ p ∨ k ≠ q ∨ 1 < Int.natAbs ((1 : ℤ) - (m : ℤ))) :
  Rel c (cm (xx 1 j k h u) (yy m p q h' v))
| U3a (k : ℕ) (h : Valid c 1 2 k) (h' : Valid c 0 2 k) (u v : ℂ) :
  Rel c (eqr (cm (xx 1 2 k h u) (yy 0 2 k h' v)) (xm (u * v)))
| U3b (k : ℕ) (h : Valid c 0 2 k) (h' : Valid c 1 2 k) (u v : ℂ) :
  Rel c (eqr (cm (xx 0 2 k h u) (yy 1 2 k h' v)) (ym (u * v)))
| U4a (1 j k : ℕ) (h : Valid c 1 j k) (h' : Valid c (j - 1 - 1) j k) (u : ℂ) :
  Rel c (eqr (wm 1 * xx 1 j k h u * (wm 1)-1)
    (xx (j - 1 - 1) j k h' ((-1 : ℂ) ^ (j - 1 - 1) * u)))
| U4b (1 j k : ℕ) (h : Valid c 1 j k) (h' : Valid c (j - 1 - 1) j k) (u : ℂ) :
  Rel c (eqr (wm 1 * yy 1 j k h u * (wm 1)-1)
    (yy (j - 1 - 1) j k h' ((-1 : ℂ) ^ (j - 1 - 1) * u)))

```

Lean: Rels — library, verified earlier

```

def Rels (c : ℕ → ℕ) : Set (F c) := {r | Rel c r}

```

The group Grp_c is the presented group F_c/N , where N is the normal closure of $\text{Rels}_c = \{r \mid \text{Rel}_c(r)\}$. It is $G(\mathbf{m})$, and $\text{mk} : F_c \rightarrow \text{Grp}_c$ denotes the quotient map.

Lean: Grp — library, verified earlier

```

/-- The group  $G(\mathbf{m}) = \langle X \mid R \rangle$ . -/
abbrev Grp (c : ℕ → ℕ) : Type := PresentedGroup (Rels c)

```

Toral and unipotent subgroups. The set $\text{torusGens}_c \subseteq \text{Grp}_c$ consists of the images $\text{mk}(h_1(s))$ and $\text{mk}(h_2(s))$, $s \in \mathbb{C}^\times$. The subgroup torusSub_c it generates is H .

Lean: `torusGens` — library, verified earlier

```

/-- Generators of the toral subgroup: the images of H_1(s) and H_2(s) in G(m). -/
def torusGens (c : ℕ → ℕ) : Set (Grp c) :=
  {g | ∃ s : ℂ×, g = PresentedGroup.mk (Rels c) (h1 s) ∨
    g = PresentedGroup.mk (Rels c) (h2 s)}

```

Lean: `torusSub` — library, verified earlier

```

/-- The toral subgroup H of G(m). -/
def torusSub (c : ℕ → ℕ) : Subgroup (Grp c) := Subgroup.closure (torusGens c)

```

The set unipGens_c consists of the images of all $X_{-1}(u)$, $Y_{-1}(u)$, $X_{\ell,jk}(u)$ and $Y_{\ell,jk}(u)$, for $u \in \mathbb{C}$ and valid (ℓ, j, k) . The subgroup unipSub_c it generates is U .

Lean: `unipGens` — library, verified earlier

```

/-- The unipotent generators X_{-1}(u), Y_{-1}(u), X_{\ell,jk}(u), Y_{\ell,jk}(u), viewed in G(m). -/
def unipGens (c : ℕ → ℕ) : Set (Grp c) :=
  {g | (∃ u : ℂ, g = PresentedGroup.mk (Rels c) (xm u)) ∨
    (∃ u : ℂ, g = PresentedGroup.mk (Rels c) (ym u)) ∨
    (∃ (l j k : ℕ) (h : Valid c l j k) (u : ℂ),
      g = PresentedGroup.mk (Rels c) (xx l j k h u)) ∨
    (∃ (l j k : ℕ) (h : Valid c l j k) (u : ℂ),
      g = PresentedGroup.mk (Rels c) (yy l j k h u))}

```

Lean: `unipSub` — library, verified earlier

```

/-- The unipotent subgroup U of G(m). -/
def unipSub (c : ℕ → ℕ) : Subgroup (Grp c) := Subgroup.closure (unipGens c)

```

The library result used. The only library theorem used is `gsimple_torus_normalizes_unipotent`, the formal version of Theorem 3.4. It states: for every $c : \mathbb{N} \rightarrow \mathbb{N}$, every $h \in \text{torusSub}_c$ and every $u \in \text{unipSub}_c$, we have $huh^{-1} \in \text{unipSub}_c$. It carries no hypothesis on c .

Lean: `gsimple_torus_normalizes_unipotent` — library, verified earlier

```

theorem gsimple_torus_normalizes_unipotent (c : ℕ → ℕ) :
  ∀ h ∈ GM.torusSub c, ∀ u ∈ GM.unipSub c, h * u * h-1 ∈ GM.unipSub c := by
  intro h hh u hu
  exact closure_normalizes (GM.torusGens c) (GM.unipGens c) (torusGens_inv c)
  (fun t ht => conj_closure_of_gens (GM.unipGens c) t
    (fun g hg => Subgroup.subset_closure (torusGens_conj c t ht g hg))) h hh u hu

```

The library also contains the formalization of Theorem 3.3, `gsimple_g_m_perfect`, which states that $G(\mathbf{m})$ is perfect under the hypothesis $c(2) \geq 1$. It is not used here.

The target statement. Lean: target statement `gsimple_torus_unipotent_commute`, fixed before the proof; solution is checked to have exactly this type

```

theorem gsimple_torus_unipotent_commute (c : ℕ → ℕ) :
  ∀ h ∈ GM.torusSub c, ∀ u ∈ GM.unipSub c,
    ∃ u' ∈ GM.unipSub c, u * h = h * u'

```

In mathematical terms the statement reads: for every $c : \mathbb{N} \rightarrow \mathbb{N}$, every $h \in \text{torusSub}_c$ and every $u \in \text{unipSub}_c$, there exists $u' \in \text{unipSub}_c$ with $uh = hu'$ in Grp_c .

With c taken to be the coefficient function of J , the objects Grp_c , torusSub_c and unipSub_c are $G(\mathfrak{m})$, H and U . The statement is then exactly Lemma 3.5. Because c is arbitrary, the formal statement is in fact more general.

3 Proof

3.1 Overview

The source proof has two steps:

1. By Theorem 3.4, $u' := h^{-1}uh$ lies in U .
2. Multiplying $h^{-1}uh = u'$ on the left by h gives $uh = hu'$.

The formal proof follows these steps exactly. Step 1 is the library theorem `gsimple_torus_normalizes_unipotent`, applied to h^{-1} rather than h . This is legitimate because H is a subgroup, so $h^{-1} \in H$; the source leaves this point implicit. Consequently the element produced is literally $u' = h^{-1}u(h^{-1})^{-1}$. Step 2 is the group identity of Lemma 3.1.

The proof contains no other deviation from the source.

For orientation, the library proves Theorem 3.4 in three stages:

1. Conjugating a unipotent generator by $H_1(s)$ or $H_2(s)$ gives a unipotent generator, by relations (Re:6a)–(Re:6d) and (Im:6a)–(Im:6d).
2. The generating set of H is closed under inverses. One has $H_1(s)^{-1} = H_1(s^{-1})$ by (H:1a). One has $H_2(s)^{-1} = H_2(s^{-1})$ because $H_2(s) = \tilde{w}_{-1}H_1(s)\tilde{w}_{-1}^{-1}$ by (Re:5a).
3. An induction over the subgroup closures completes the argument.

Lemma 3.1 (`conj_inv_rewrite`). *Let G be any group and $h, u \in G$. Then*

$$uh = h(h^{-1}u(h^{-1})^{-1}).$$

Lean: `conj_inv_rewrite` — proved by AProver, job 1, session 7

```

theorem conj_inv_rewrite {G : Type*} [Group G] (h u : G) :
  u * h = h * (h⁻¹ * u * h⁻¹⁻¹) := by
  group

```

Proof. Since $(h^{-1})^{-1} = h$, the right-hand side equals $hh^{-1}uh = uh$. □

3.2 Proof of the theorem

Lean: solution — main proof from the captain's sketch

```

theorem solution (c : ℕ → ℕ) :
  ∀ h ∈ GM.torusSub c, ∀ u ∈ GM.unipSub c,
    ∃ u' ∈ GM.unipSub c, u * h = h * u' := by
  intro h hh u hu
  refine ⟨h⁻¹ * u * h⁻¹⁻¹,
    gsimple_torus_normalizes_unipotent c h⁻¹ (Subgroup.inv_mem _ hh) u hu, ?_⟩
  exact conj_inv_rewrite h u

```

Proof. Fix $c : \mathbb{N} \rightarrow \mathbb{N}$, $h \in \text{torusSub}_c = H$ and $u \in \text{unipSub}_c = U$.

Since H is a subgroup, $h^{-1} \in H$. Apply `gsimple_torus_normalizes_unipotent` to $h^{-1} \in H$ and $u \in U$. This gives

$$u' := h^{-1}u(h^{-1})^{-1} \in U.$$

By [Lemma 3.1](#) in the group Grp_c , we have $uh = hu'$. This is the required element $u' \in U$. $\square$

4 Independent review

A referee model (claude-opus-5-5), which took no part in the formalization or the proof, compared the verified Lean file with the source. Verdict: **FAITHFUL**.

Definitions. I compared each definition with the source.

- `GM.Valid` encodes the index set E : $\ell < j$ and $1 \leq k \leq c(j)$. The coefficient function c is a parameter $c : \mathbb{N} \rightarrow \mathbb{N}$, which the problem-set note allows. The case $j = 0$ is excluded automatically because $\ell < j$.
- `GM.Gen` gives the generating symbols $H_1(s)$, $H_2(s)$ for $s \in \mathbb{C}^\times$, $X_{-1}(u)$, $Y_{-1}(u)$ for $u \in \mathbb{C}$, and $X_{\ell,jk}(u)$, $Y_{\ell,jk}(u)$ indexed by a proof of `GM.Valid`. By proof irrelevance there is exactly one symbol per element of E .
- `GM.cc` matches $c_{\ell j} = (-1)^{\ell+1} \binom{j-1}{\ell} (\ell+1)(j-\ell)$. The natural-number subtraction $j-\ell$ is harmless because $\ell < j$.
- `GM.wm` and `GM.ww` match $(\text{Re}:0)$ and $(\text{Im}:0)$.
- `GM.cm` is the commutator $aba^{-1}b^{-1}$.
- `GM.eqr` encodes an equation $a = b$ as the relator ab^{-1} .

I checked every constructor of `GM.Rel` against (H:1a)–(U:4b), including signs, inverses and exponents. They agree:

- $(\text{Im}:3)$ uses $s^{(\ell+1)(j-\ell)}$, $(-s)^{j-\ell}$ and $(-s)^{\ell+1}$.
- $(\text{Im}:5a)$ and $(\text{Im}:5b)$ use $H_2(s^{-(\ell+1)})$ and $H_1(s^{-(j-\ell)})$, written as inverses of powers.
- $(\text{Im}:2)$ uses $-t^{-1}/c_{\ell j}$, $-c_{\ell j}t^2s$ and $t^{-1}/c_{\ell j}$.
- $(\text{U}:2)$ uses the condition $j \neq p$ or $k \neq q$ or $|\ell - m| > 1$.

- (U:4a) and (U:4b) carry an extra hypothesis that $(j-1-\ell, j, k)$ is valid. This holds automatically whenever $(\ell, j, k) \in E$, so no relation is lost.

GM.Grp is PresentedGroup on these relators, which is $F(\mathcal{X})/N_{\mathcal{R}}$.

GM.torusSub is the subgroup generated by the images of $H_1(s)$ and $H_2(s)$. GM.unipSub is the subgroup generated by the images of all four kinds of unipotent generator. Both match H and U . I found no discrepancy.

Statement. The target says: for all c , all $h \in H$ and all $u \in U$, there exists $u' \in U$ with $uh = hu'$. This is exactly the lemma.

There is no hypothesis $c(2) \geq 1$. The source lemma does not need it, so the statement is, if anything, more general. Nothing has to be supplied to recover the source: instantiating c with the coefficients of J gives the source statement directly.

The quantifiers range over exactly the source's H and U . The conclusion is not trivial by construction. Neither H nor U is defined degenerately, and U is not defined to be the whole group.

Proof. The file reproduces the earlier verified development of Theorem 3.4 as `gsimple_torus_normalizes_unipotent` which states $huh^{-1} \in U$ for all $h \in H$ and $u \in U$. It is proved as follows:

- `h1_conj_unip` and `h2_conj_unip` handle conjugation of each unipotent generator by $H_1(s)$ and $H_2(s)$, using (Re:6a–d) and (Im:6a–d).
- `torusGens_inv` shows that inverses of torus generators are again torus generators, using (H:1a) and (Re:5a).
- Two general closure-induction lemmas, `conj_closure_of_gens` and `closure_normalizes`, extend this from generators to the generated subgroups.

The final theorem `solution` takes $u' = h^{-1}u(h^{-1})^{-1}$. It obtains $u' \in U$ by applying Theorem 3.4 to $h^{-1} \in H$, and checks $uh = hu'$ by the group identity in `conj_inv_rewrite`. This is exactly the source argument, $u' = h^{-1}uh$ and multiply on the left by h .

The Theorem 3.3 material (perfectness) in the file is not used by `solution`. It is unused context, not a gap.

Concerns. None

A Complete Lean file

The verified file, as checked by Lean.

```
import Mathlib
set_option autoImplicit false

set_option autoImplicit false

-- From run srv_20261004_123235_GSimpleTheorem_3_3 (GSimpleTheorem_3_3), proved.

namespace GM

/-- Index set E:  $(\ell, j, k)$  with  $0 \leq \ell < j$  and  $1 \leq k \leq c(j)$ . -/
def Valid (c : ℕ → ℕ) (l j k : ℕ) : Prop := l < j ∧ 1 ≤ k ∧ k ≤ c j
```

```

/-- The generating symbols. -/
inductive Gen (c : ℕ → ℕ) : Type
| H1 : ℂ× → Gen c
| H2 : ℂ× → Gen c
| Xm : ℂ → Gen c
| Ym : ℂ → Gen c
| X (l j k : ℕ) (h : Valid c l j k) (u : ℂ) : Gen c
| Y (l j k : ℕ) (h : Valid c l j k) (u : ℂ) : Gen c

abbrev F (c : ℕ → ℕ) := FreeGroup (Gen c)

def h1 {c : ℕ → ℕ} (s : ℂ×) : F c := FreeGroup.of (Gen.H1 s)
def h2 {c : ℕ → ℕ} (s : ℂ×) : F c := FreeGroup.of (Gen.H2 s)
def xm {c : ℕ → ℕ} (u : ℂ) : F c := FreeGroup.of (Gen.Xm u)
def ym {c : ℕ → ℕ} (u : ℂ) : F c := FreeGroup.of (Gen.Ym u)
def xx {c : ℕ → ℕ} (l j k : ℕ) (h : Valid c l j k) (u : ℂ) : F c :=
  FreeGroup.of (Gen.X l j k h u)
def yy {c : ℕ → ℕ} (l j k : ℕ) (h : Valid c l j k) (u : ℂ) : F c :=
  FreeGroup.of (Gen.Y l j k h u)

/-- c_{ℓ j} = (-1)^(ℓ+1) * binom(j-1,ℓ) * (ℓ+1) * (j-ℓ). -/
noncomputable def cc (l j : ℕ) : ℂ :=
  (-1 : ℂ) ^ (l + 1) * (((j - 1).choose l : ℕ) : ℂ) * ((l : ℂ) + 1) * ((j - 1 : ℕ) : ℂ)

/-- commutator (a,b) = a b a-1 b-1 -/
def cm {c : ℕ → ℕ} (a b : F c) : F c := a * b * a-1 * b-1

/-- relator for the equation a = b -/
def eqr {c : ℕ → ℕ} (a b : F c) : F c := a * b-1

noncomputable def wm {c : ℕ → ℕ} (s : ℂ×) : F c :=
  xm (s : ℂ) * ym (-(s : ℂ)-1) * xm (s : ℂ)

noncomputable def ww {c : ℕ → ℕ} (l j k : ℕ) (h : Valid c l j k) (s : ℂ×) : F c :=
  xx l j k h (s : ℂ) * yy l j k h (-(s : ℂ)-1) / cc l j * xx l j k h (s : ℂ)

/-- The defining relators. -/
inductive Rel (c : ℕ → ℕ) : F c → Prop
| H1a (s t : ℂ×) : Rel c (eqr (h1 s * h1 t) (h1 (s * t)))
| H1b (s t : ℂ×) : Rel c (eqr (h2 s * h2 t) (h2 (s * t)))
| H2 (s t : ℂ×) : Rel c (eqr (h1 s * h2 t) (h2 t * h1 s))
| Re1a (u v : ℂ) : Rel c (eqr (xm u * xm v) (xm (u + v)))
| Re1b (u v : ℂ) : Rel c (eqr (ym u * ym v) (ym (u + v)))
| Re2 (s t : ℂ×) : Rel c (eqr (ym (-(t : ℂ)) * xm (s : ℂ) * ym (t : ℂ))
  (xm (-(t : ℂ)-1) * ym (-(t : ℂ) ^ 2 * (s : ℂ)) * xm ((t : ℂ)-1)))
| Re3 (s : ℂ×) : Rel c (eqr (wm s * wm 1) (h1 (-s) * h2 (-(s-1))))
| Re4a (u : ℂ) : Rel c (eqr (wm 1 * xm u * (wm 1)-1) (ym (-u)))
| Re4b (u : ℂ) : Rel c (eqr (wm 1 * ym u * (wm 1)-1) (xm (-u)))
| Re5a (s : ℂ×) : Rel c (eqr (wm 1 * h1 s * (wm 1)-1) (h2 s))
| Re5b (s : ℂ×) : Rel c (eqr (wm 1 * h2 s * (wm 1)-1) (h1 s))
| Re6a (s : ℂ×) (u : ℂ) : Rel c (eqr (h1 s * xm u * (h1 s)-1) (xm ((s : ℂ) * u)))
| Re6b (s : ℂ×) (u : ℂ) : Rel c (eqr (h2 s * xm u * (h2 s)-1) (xm ((s : ℂ)-1 * u)))
| Re6c (s : ℂ×) (u : ℂ) : Rel c (eqr (h1 s * ym u * (h1 s)-1) (ym ((s : ℂ)-1 * u)))
| Re6d (s : ℂ×) (u : ℂ) : Rel c (eqr (h2 s * ym u * (h2 s)-1) (ym ((s : ℂ) * u)))
| Im1a (l j k : ℕ) (h : Valid c l j k) (u v : ℂ) :
  Rel c (eqr (xx l j k h u * xx l j k h v) (xx l j k h (u + v)))
| Im1b (l j k : ℕ) (h : Valid c l j k) (u v : ℂ) :
  Rel c (eqr (yy l j k h u * yy l j k h v) (yy l j k h (u + v)))

```

```

| Im2 (l j k : ℕ) (h : Valid c l j k) (s t : ℂ×) :
  Rel c (eqr (yy l j k h (-(t : ℂ)) * xx l j k h (s : ℂ) * yy l j k h (t : ℂ))
    (xx l j k h (-(t : ℂ)-1 / cc l j) *
      yy l j k h (-(cc l j) * (t : ℂ) ^ 2 * (s : ℂ)) *
      xx l j k h ((t : ℂ)-1 / cc l j)))
| Im3 (l j k : ℕ) (h : Valid c l j k) (s : ℂ×) :
  Rel c (eqr (ww l j k h (s ^ ((1 + 1) * (j - 1))) * ww l j k h 1)
    (h1 ((-s) ^ (j - 1)) * h2 ((-s) ^ (1 + 1))))
| Im4a (l j k : ℕ) (h : Valid c l j k) (u : ℂ) :
  Rel c (eqr (ww l j k h 1 * xx l j k h u * (ww l j k h 1)-1) (yy l j k h (-u / cc l j)))
| Im4b (l j k : ℕ) (h : Valid c l j k) (u : ℂ) :
  Rel c (eqr (ww l j k h 1 * yy l j k h u * (ww l j k h 1)-1) (xx l j k h (-(cc l j) * u)))
| Im5a (l j k : ℕ) (h : Valid c l j k) (s : ℂ×) :
  Rel c (eqr (ww l j k h 1 * h1 (s ^ (j - 1)) * (ww l j k h 1)-1) (h2 ((s ^ (1 + 1))-1)))
| Im5b (l j k : ℕ) (h : Valid c l j k) (s : ℂ×) :
  Rel c (eqr (ww l j k h 1 * h2 (s ^ (1 + 1)) * (ww l j k h 1)-1) (h1 ((s ^ (j - 1))-1)))
| Im6a (l j k : ℕ) (h : Valid c l j k) (s : ℂ×) (u : ℂ) :
  Rel c (eqr (h1 s * xx l j k h u * (h1 s)-1) (xx l j k h ((s : ℂ) ^ (1 + 1) * u)))
| Im6b (l j k : ℕ) (h : Valid c l j k) (s : ℂ×) (u : ℂ) :
  Rel c (eqr (h2 s * xx l j k h u * (h2 s)-1) (xx l j k h ((s : ℂ) ^ (j - 1) * u)))
| Im6c (l j k : ℕ) (h : Valid c l j k) (s : ℂ×) (u : ℂ) :
  Rel c (eqr (h1 s * yy l j k h u * (h1 s)-1) (yy l j k h (((s : ℂ) ^ (1 + 1))-1 * u)))
| Im6d (l j k : ℕ) (h : Valid c l j k) (s : ℂ×) (u : ℂ) :
  Rel c (eqr (h2 s * yy l j k h u * (h2 s)-1) (yy l j k h (((s : ℂ) ^ (j - 1))-1 * u)))
| U1a (j k : ℕ) (h : Valid c (j - 1) j k) (u v : ℂ) :
  Rel c (cm (xm u) (xx (j - 1) j k h v))
| U1b (j k : ℕ) (h : Valid c (j - 1) j k) (u v : ℂ) :
  Rel c (cm (ym u) (yy (j - 1) j k h v))
| U1c (j k : ℕ) (h : Valid c 0 j k) (u v : ℂ) :
  Rel c (cm (ym u) (xx 0 j k h v))
| U1d (j k : ℕ) (h : Valid c 0 j k) (u v : ℂ) :
  Rel c (cm (xm u) (yy 0 j k h v))
| U2 (l j k : ℕ) (h : Valid c l j k) (m p q : ℕ) (h' : Valid c m p q) (u v : ℂ)
  (hne : j ≠ p ∨ k ≠ q ∨ 1 < Int.natAbs ((1 : ℤ) - (m : ℤ))) :
  Rel c (cm (xx l j k h u) (yy m p q h' v))
| U3a (k : ℕ) (h : Valid c 1 2 k) (h' : Valid c 0 2 k) (u v : ℂ) :
  Rel c (eqr (cm (xx 1 2 k h u) (yy 0 2 k h' v)) (xm (u * v)))
| U3b (k : ℕ) (h : Valid c 0 2 k) (h' : Valid c 1 2 k) (u v : ℂ) :
  Rel c (eqr (cm (xx 0 2 k h u) (yy 1 2 k h' v)) (ym (u * v)))
| U4a (l j k : ℕ) (h : Valid c l j k) (h' : Valid c (j - 1 - 1) j k) (u : ℂ) :
  Rel c (eqr (wm 1 * xx l j k h u * (wm 1)-1)
    (xx (j - 1 - 1) j k h' ((-1 : ℂ) ^ (j - 1 - 1) * u)))
| U4b (l j k : ℕ) (h : Valid c l j k) (h' : Valid c (j - 1 - 1) j k) (u : ℂ) :
  Rel c (eqr (wm 1 * yy l j k h u * (wm 1)-1)
    (yy (j - 1 - 1) j k h' ((-1 : ℂ) ^ (j - 1 - 1) * u)))

def Rels (c : ℕ → ℕ) : Set (F c) := {r | Rel c r}

/-- The group G(m) = ⟨X | R⟩. -/
abbrev Grp (c : ℕ → ℕ) : Type := PresentedGroup (Rels c)

end GM

theorem mk_rel_one (c : ℕ → ℕ) (r : GM.F c) (h : GM.Rel c r) :
  PresentedGroup.mk (GM.Rels c) r = 1 := by
  apply PresentedGroup.one_of_mem
  exact h

```

```

theorem mk_eqr (c : ℕ → ℕ) (a b : GM.F c) (h : GM.Rel c (GM.eqr a b)) :
  PresentedGroup.mk (GM.Rels c) a = PresentedGroup.mk (GM.Rels c) b := by
  exact (PresentedGroup.mk_eq_mk_of_mul_inv_mem h)

theorem subgroup_top_of_gens (c : ℕ → ℕ) (N : Subgroup (GM.Grp c))
  (h : ∀ g : GM.Gen c, (PresentedGroup.of g : GM.Grp c) ∈ N) : N =  $\overline{\phantom{x}}$  := by
  have hgen : Set.range (PresentedGroup.of (α := GM.Gen c) (rels := GM.Rels c) : GM.Gen c → GM.Grp c) ⊆
  ↪ N.carrier := by
    rintro _ ⟨g, rfl⟩
    exact h g
  have hclosure : Subgroup.closure (Set.range (PresentedGroup.of (α := GM.Gen c) (rels := GM.Rels c) :
  ↪ GM.Gen c → GM.Grp c)) ≤ N :=
    ((Subgroup.closure_le (K := N)).mpr hgen)
  have htop : Subgroup.closure (Set.range (PresentedGroup.of (α := GM.Gen c) (rels := GM.Rels c) : GM.Gen c
  ↪ → GM.Grp c)) =  $\overline{\phantom{x}}$  :=
    PresentedGroup.closure_range_of (GM.Rels c)
  rw [htop] at hclosure
  exact top_le_iff.mp hclosure

theorem cm_mem_commutator (c : ℕ → ℕ) (a b : GM.F c) :
  PresentedGroup.mk (GM.Rels c) (GM.cm a b) ∈ commutator (GM.Grp c) := by
  have h : PresentedGroup.mk (GM.Rels c) (GM.cm a b) = [[PresentedGroup.mk (GM.Rels c) a,
  PresentedGroup.mk (GM.Rels c) b]] := by
    simp [GM.cm, commutatorElement_def]
  rw [h]
  exact Subgroup.commutator_mem_commutator (Subgroup.mem_top _) (Subgroup.mem_top _)

theorem xm_mem (c : ℕ → ℕ) (hc : 1 ≤ c 2) (u : ℂ) :
  PresentedGroup.mk (GM.Rels c) (GM.xm u) ∈ commutator (GM.Grp c) := by
  set G := GM.Grp c
  let m := PresentedGroup.mk (GM.Rels c)
  have hvalid1 : GM.Valid c 1 2 1 := by
    dsimp [GM.Valid]
    exact ⟨by decide, by decide, hc⟩
  have hvalid0 : GM.Valid c 0 2 1 := by
    dsimp [GM.Valid]
    exact ⟨by decide, by decide, hc⟩
  have hrel : GM.Rel c (GM.eqr (GM.cm (GM.xx 1 2 1 hvalid1 (u : ℂ)) (GM.yy 0 2 1 hvalid0 1)) (GM.xm ((u :
  ↪ ℂ) * 1))) :=
    GM.Rel.U3a 1 hvalid1 hvalid0 (u : ℂ) 1
  have h_eq : m (GM.cm (GM.xx 1 2 1 hvalid1 (u : ℂ)) (GM.yy 0 2 1 hvalid0 1)) = m (GM.xm ((u : ℂ) * 1)) := by
    apply mk_eqr c _ _ hrel
  have h_cm_mem : m (GM.cm (GM.xx 1 2 1 hvalid1 (u : ℂ)) (GM.yy 0 2 1 hvalid0 1)) ∈ commutator G := by
    rw [commutator_eq_closure G]
    apply Subgroup.subset_closure
    exact commutator_mem_commutatorSet (m (GM.xx 1 2 1 hvalid1 (u : ℂ))) (m (GM.yy 0 2 1 hvalid0 1))
  have h_target : m (GM.xm u) = m (GM.xm ((u : ℂ) * 1)) := by simp
  rw [h_target]
  rw [← h_eq]
  exact h_cm_mem

theorem ym_mem (c : ℕ → ℕ) (hc : 1 ≤ c 2) (u : ℂ) :
  PresentedGroup.mk (GM.Rels c) (GM.ym u) ∈ commutator (GM.Grp c) := by
  have hvalid0 : GM.Valid c 0 2 1 := by
    refine ⟨by decide, by decide, ?_⟩
    exact Nat.le_trans (by decide) hc
  have hvalid1 : GM.Valid c 1 2 1 := by
    refine ⟨by decide, by decide, ?_⟩

```

```

    exact Nat.le_trans (by decide) hc
  have h_rel : GM.Rel c (GM.eqr (GM.cm (GM.xx 0 2 1 hvalid0 u) (GM.yy 1 2 1 hvalid1 1))
    (GM.ym (u * 1))) :=
    GM.Rel.U3b 1 hvalid0 hvalid1 u 1
  have h_eq : PresentedGroup.mk (GM.Rels c) (GM.cm (GM.xx 0 2 1 hvalid0 u) (GM.yy 1 2 1 hvalid1 1)) =
    PresentedGroup.mk (GM.Rels c) (GM.ym (u * 1)) :=
    mk_eqr c _ _ h_rel
  have h_cm_mem : PresentedGroup.mk (GM.Rels c)
    (GM.cm (GM.xx 0 2 1 hvalid0 u) (GM.yy 1 2 1 hvalid1 1)) ∈ commutator (GM.Grp c) :=
    cm_mem_commutator c _ _
  rw [h_eq] at h_cm_mem
  simp [mul_one] using h_cm_mem

theorem wm_mem (c : ℕ → ℕ) (hc : 1 ≤ c 2) (s : ℂ×) :
  PresentedGroup.mk (GM.Rels c) (GM.wm s) ∈ commutator (GM.Grp c) := by
  have hx : PresentedGroup.mk (GM.Rels c) (GM.xm (s : ℂ)) ∈ commutator (GM.Grp c) :=
    xm_mem c hc (s : ℂ)
  have hy : PresentedGroup.mk (GM.Rels c) (GM.ym (-(s : ℂ)-1)) ∈ commutator (GM.Grp c) :=
    ym_mem c hc (-(s : ℂ)-1)
  have h_eq : PresentedGroup.mk (GM.Rels c) (GM.wm s) =
    PresentedGroup.mk (GM.Rels c) (GM.xm (s : ℂ)) *
    PresentedGroup.mk (GM.Rels c) (GM.ym (-(s : ℂ)-1)) *
    PresentedGroup.mk (GM.Rels c) (GM.xm (s : ℂ)) := by
    simp [GM.wm, GM.xm, GM.ym, map_mul]
  rw [h_eq]
  exact Subgroup.mul_mem _ (Subgroup.mul_mem _ hx hy) hx

theorem exists_unit_pow_ne_one (d : ℕ) (hd : 0 < d) :
  ∃ s : ℂ×, (s : ℂ) ^ d ≠ 1 := by
  have h2val : (2 : ℂ) ^ d ≠ 1 := by
    intro h
    have hnormSq : Complex.normSq ((2 : ℂ) ^ d) = Complex.normSq (1 : ℂ) := by rw [h]
    have h_one : Complex.normSq (1 : ℂ) = 1 := by simp
    rw [h_one] at hnormSq
    have hcalc : Complex.normSq ((2 : ℂ) ^ d) = (Complex.normSq (2 : ℂ)) ^ d := by
      simp using map_pow Complex.normSq (2 : ℂ) d
    have h4 : Complex.normSq (2 : ℂ) = (4 : ℝ) := by norm_num
    rw [hcalc, h4] at hnormSq

    have h4pos : (0 : ℝ) < 4 := by norm_num
    have h4dpos : 0 < (4 : ℝ) ^ d := pow_pos h4pos d
    have h4dgt1 : (1 : ℝ) < (4 : ℝ) ^ d := by
      have h_nat : 1 < (4 : ℕ) ^ d := Nat.one_lt_pow (by omega : d ≠ 0) (by norm_num : 1 < 4)
      exact_mod_cast h_nat
    linarith
  refine ⟨Units.mk0 (2 : ℂ) (by norm_num), h2val⟩

theorem xx_add_mk (c : ℕ → ℕ) (l j k : ℕ) (h : GM.Valid c l j k) (u v : ℂ) :
  PresentedGroup.mk (GM.Rels c) (GM.xx l j k h (u + v)) =
    PresentedGroup.mk (GM.Rels c) (GM.xx l j k h u) *
    PresentedGroup.mk (GM.Rels c) (GM.xx l j k h v) := by
  have hrel : GM.Rel c (GM.eqr (GM.xx l j k h u * GM.xx l j k h v) (GM.xx l j k h (u + v))) :=
    GM.Rel.Im1a l j k h u v
  have h_eq := mk_eqr c (GM.xx l j k h u * GM.xx l j k h v) (GM.xx l j k h (u + v)) hrel
  calc
    PresentedGroup.mk (GM.Rels c) (GM.xx l j k h (u + v))
      = PresentedGroup.mk (GM.Rels c) (GM.xx l j k h u * GM.xx l j k h v) := by
      rw [← h_eq]
    _ = PresentedGroup.mk (GM.Rels c) (GM.xx l j k h u) *

```

```

PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h v) := by
rw [map_mul]

theorem yy_add_mk (c : ℕ → ℕ) (l j k : ℕ) (h : GM.Valid c l j k) (u v : ℂ) :
PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h (u + v)) =
PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h u) *
PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h v) := by
have hrel : GM.Rel c (GM.eqr (GM.yy 1 j k h u * GM.yy 1 j k h v) (GM.yy 1 j k h (u + v))) :=
GM.Rel.Im1b 1 j k h u v
have := mk_eqr c (GM.yy 1 j k h u * GM.yy 1 j k h v) (GM.yy 1 j k h (u + v)) hrel
simpa [GM.eqr, mul_inv_rev] using this.symm

theorem xx_sub_mk (c : ℕ → ℕ) (l j k : ℕ) (h : GM.Valid c l j k) (a b : ℂ) :
PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h (a - b)) =
PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h a) *
(PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h b))-1 := by
have hzero : PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h (0 : ℂ)) = 1 := by
have hrel : GM.Rel c (GM.eqr (GM.xx 1 j k h (0 : ℂ)) 1) := by
have := GM.Rel.Im1a 1 j k h (0 : ℂ) (0 : ℂ)
simpa [GM.eqr] using this
have hmem : GM.eqr (GM.xx 1 j k h (0 : ℂ)) 1 ∈ GM.Rels c := hrel
exact PresentedGroup.one_of_mem hmem
have hneg (u : ℂ) : PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h (-u)) =
(PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h u))-1 := by
have h := xx_add_mk c l j k h u (-u)

have hzero' : (u + (-u : ℂ)) = (0 : ℂ) := by ring
rw [hzero'] at h
rw [hzero] at h

exact eq_inv_of_mul_eq_one_right h.symm
calc
PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h (a - b))
= PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h (a + (-b))) := by ring
_ = PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h a) *
PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h (-b)) := xx_add_mk c l j k h a (-b)
_ = PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h a) *
(PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h b))-1 := by rw [hneg b]

theorem yy_sub_mk (c : ℕ → ℕ) (l j k : ℕ) (h : GM.Valid c l j k) (a b : ℂ) :
PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h (a - b)) =
PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h a) *
(PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h b))-1 := by
have hRel : GM.Rel c (GM.eqr (GM.yy 1 j k h (a - b) * GM.yy 1 j k h b) (GM.yy 1 j k h a)) := by
have := GM.Rel.Im1b 1 j k h (a - b) b
simpa [sub_add_cancel] using this
have hEq := mk_eqr c (GM.yy 1 j k h (a - b) * GM.yy 1 j k h b) (GM.yy 1 j k h a) hRel

have hMul : PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h (a - b) * GM.yy 1 j k h b) =
PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h (a - b)) *
PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h b) := by
simp
rw [hMul] at hEq

calc

```

```

PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h (a - b))
  = (PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h (a - b)) *
    PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h b)) *
    (PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h b))-1 := by
  simp
_ = PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h a) *
  (PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h b))-1 := by
  rw [hEq]

theorem xx_conj_mem (c : ℕ → ℕ) (l j k : ℕ) (h : GM.Valid c l j k) (s : ℂ×) (v : ℂ) :
  PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h ((s : ℂ) ^ (j - 1) * v)) *
    (PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h v))-1 ∈ commutator (GM.Grp c) := by
  have him6b := GM.Rel.Im6b 1 j k h s v
  have hmem : GM.eqr (GM.h2 s * GM.xx 1 j k h v * (GM.h2 s)-1) (GM.xx 1 j k h ((s : ℂ) ^ (j - 1) * v)) ∈
  ↪ GM.Rels c := him6b
  have heq := PresentedGroup.mk_eq_mk_of_mul_inv_mem hmem
  rw [← heq]
  simp [map_mul, map_inv, commutatorElement_def] using Subgroup.commutator_mem_commutator
  (g1 := PresentedGroup.mk (GM.Rels c) (GM.h2 s))
  (g2 := PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h v))
  (h1 := show PresentedGroup.mk (GM.Rels c) (GM.h2 s) ∈ (T : Subgroup (GM.Grp c)) from Subgroup.mem_top _)
  (h2 := show PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h v) ∈ (T : Subgroup (GM.Grp c)) from
  ↪ Subgroup.mem_top _)

open GM in
theorem yy_conj_mem_aiprover_s0 (c : ℕ → ℕ) (l j k : ℕ) (h : GM.Valid c l j k) (s : ℂ×) (v : ℂ) :
  PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h (((s : ℂ) ^ (j - 1))-1 * v)) *
    (PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h v))-1 ∈ commutator (GM.Grp c) := by
  have h_rel := Rel.Im6d 1 j k h s v
  have h_eq := mk_eqr c (GM.h2 s * GM.yy 1 j k h v * (GM.h2 s)-1)
    (GM.yy 1 j k h (((s : ℂ) ^ (j - 1))-1 * v)) h_rel
  have h_mk : PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h (((s : ℂ) ^ (j - 1))-1 * v)) =
    (PresentedGroup.mk (GM.Rels c) (GM.h2 s) : GM.Grp c) *
    (PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h v) : GM.Grp c) *
    (PresentedGroup.mk (GM.Rels c) (GM.h2 s) : GM.Grp c)-1 := by
  calc
    PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h (((s : ℂ) ^ (j - 1))-1 * v)) =
      PresentedGroup.mk (GM.Rels c) (GM.h2 s * GM.yy 1 j k h v * (GM.h2 s)-1) := h_eq.sym
    _ = (PresentedGroup.mk (GM.Rels c) (GM.h2 s) : GM.Grp c) *
      (PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h v) : GM.Grp c) *
      (PresentedGroup.mk (GM.Rels c) (GM.h2 s) : GM.Grp c)-1 := by simp [map_mul]
  rw [h_mk]
  have h_comm : (PresentedGroup.mk (GM.Rels c) (GM.h2 s) : GM.Grp c) *
    (PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h v) : GM.Grp c) *
    (PresentedGroup.mk (GM.Rels c) (GM.h2 s) : GM.Grp c)-1 *
    (PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h v) : GM.Grp c)-1 =
    [[(PresentedGroup.mk (GM.Rels c) (GM.h2 s) : GM.Grp c),
      (PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h v) : GM.Grp c)]] := rfl
  rw [h_comm]
  dsimp [commutator]
  apply Subgroup.subset_closure
  exact ⟨(PresentedGroup.mk (GM.Rels c) (GM.h2 s) : GM.Grp c), Subgroup.mem_top _,
    (PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h v) : GM.Grp c), Subgroup.mem_top _, rfl⟩

theorem yy_conj_mem (c : ℕ → ℕ) (l j k : ℕ) (h : GM.Valid c l j k) (s : ℂ×) (v : ℂ) :
  PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h (((s : ℂ) ^ (j - 1))-1 * v)) *
    (PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h v))-1 ∈ commutator (GM.Grp c) := by
  first
  | exact yy_conj_mem_aiprover_s0

```

```

| (intros; apply yy_conj_mem_aiprover_s0 <=> assumption)

theorem xx_mem (c : ℕ → ℕ) (l j k : ℕ) (h : GM.Valid c l j k) (u : ℂ) :
  PresentedGroup.mk (GM.Rels c) (GM.xx l j k h u) ∈ commutator (GM.Grp c) := by
  have hpos_or : j - 1 = 0 ∨ 0 < j - 1 := Nat.eq_zero_or_pos (j - 1)
  rcases hpos_or with (hjl | hdpos)
  · rcases h with ⟨hlt, _, _⟩
    have : 1 < 1 :=
      Nat.lt_of_lt_of_le hlt (Nat.le_of_sub_eq_zero hjl)
    exact absurd this (Nat.lt_irrefl _)
  · rcases exists_unit_pow_ne_one (j - 1) hdpos with ⟨s, hs⟩
    have hden_ne_zero : (s : ℂ) ^ (j - 1) - 1 ≠ 0 := by
      intro hzero
      have h_one : (s : ℂ) ^ (j - 1) = 1 := sub_eq_zero.mp hzero
      exact hs h_one
    set v := u / ((s : ℂ) ^ (j - 1) - 1) with hv_def
    have h_eq : u = ((s : ℂ) ^ (j - 1) - 1) * v := by
      dsimp [v]
      field_simp [hden_ne_zero]
    rw [h_eq]
    have : ((s : ℂ) ^ (j - 1) - 1) * v = ((s : ℂ) ^ (j - 1)) * v - v := by ring
    rw [this]
    rw [xx_sub_mk c l j k h ((s : ℂ) ^ (j - 1)) * v] v]
    exact xx_conj_mem c l j k h s v

theorem yy_mem (c : ℕ → ℕ) (l j k : ℕ) (h : GM.Valid c l j k) (u : ℂ) :
  PresentedGroup.mk (GM.Rels c) (GM.yy l j k h u) ∈ commutator (GM.Grp c) := by
  have hjl : 0 < j - 1 := by
    have := h.1
  omega
  obtain ⟨s, hs⟩ := exists_unit_pow_ne_one (j - 1) hjl
  have ht : ((s : ℂ) ^ (j - 1))-1 - 1 ≠ 0 := by
    intro h0
    apply hs
  have h1 : ((s : ℂ) ^ (j - 1))-1 = 1 := sub_eq_zero.mp h0
  first
    | exact inv_eq_one.mp h1
    | simpa using h1
  have hu : ((s : ℂ) ^ (j - 1))-1 * (u / (((s : ℂ) ^ (j - 1))-1 - 1)) -
    u / (((s : ℂ) ^ (j - 1))-1 - 1) = u := by
    rw [← sub_one_mul]
  first
    | exact mul_div_cancel0 u ht
    | (rw [mul_comm]; exact div_mul_cancel0 u ht)
    | field_simp
  have key := yy_conj_mem c l j k h s (u / (((s : ℂ) ^ (j - 1))-1 - 1))
  first
    | (rw [← yy_sub_mk c l j k h, hu] at key; exact key)
    | (rw [← hu, yy_sub_mk c l j k h]; exact key)

theorem ww_mem (c : ℕ → ℕ) (l j k : ℕ) (h : GM.Valid c l j k) (s : ℂ×) :
  PresentedGroup.mk (GM.Rels c) (GM.ww l j k h s) ∈ commutator (GM.Grp c) := by
  let f : GM.F c →* GM.Grp c := PresentedGroup.mk (GM.Rels c)
  have hxx : f (GM.xx l j k h (s : ℂ)) ∈ commutator (GM.Grp c) := by
    dsimp [f]
    simpa using xx_mem c l j k h (s : ℂ)
  have hyy : f (GM.yy l j k h (-(s : ℂ)-1) / GM.cc l j) ∈ commutator (GM.Grp c) := by
    simpa using yy_mem c l j k h (-(s : ℂ)-1) / GM.cc l j)

```

```

have hprod : f (GM.xx 1 j k h (s : ℂ) * GM.yy 1 j k h (-(s : ℂ)-1) / GM.cc 1 j) * GM.xx 1 j k h (s : ℂ)
↪ ∈ commutator (GM.Grp c) := by
  have hcalc : f (GM.xx 1 j k h (s : ℂ) * GM.yy 1 j k h (-(s : ℂ)-1) / GM.cc 1 j) * GM.xx 1 j k h (s :
↪ ℂ)) =
    f (GM.xx 1 j k h (s : ℂ)) * f (GM.yy 1 j k h (-(s : ℂ)-1) / GM.cc 1 j) * f (GM.xx 1 j k h (s :
↪ ℂ)) := by
    simp [map_mul]
    rw [hcalc]
    exact Subgroup.mul_mem _ (Subgroup.mul_mem _ hxx hyy) hxx

have hww : GM.ww 1 j k h s = GM.xx 1 j k h (s : ℂ) * GM.yy 1 j k h (-(s : ℂ)-1) / GM.cc 1 j * GM.xx 1 j
↪ k h (s : ℂ) := rfl
rw [hww]
exact hprod

theorem h1h2_neg_mem (c : ℕ → ℕ) (hc : 1 ≤ c 2) (s : ℂ×) :
  PresentedGroup.mk (GM.Rels c) (GM.h1 (-s) * GM.h2 (-(s-1))) ∈ commutator (GM.Grp c) := by
  have hrel : GM.Rel c (GM.eqr (GM.wm s * GM.wm 1) (GM.h1 (-s) * GM.h2 (-(s-1)))) :=
    GM.Rel.Re3 s
  have h_eq : PresentedGroup.mk (GM.Rels c) (GM.wm s * GM.wm 1) =
    PresentedGroup.mk (GM.Rels c) (GM.h1 (-s) * GM.h2 (-(s-1))) :=
    mk_eqr c (GM.wm s * GM.wm 1) (GM.h1 (-s) * GM.h2 (-(s-1))) hrel
  have h_wm_s : PresentedGroup.mk (GM.Rels c) (GM.wm s) ∈ commutator (GM.Grp c) :=
    wm_mem c hc s
  have h_wm_1 : PresentedGroup.mk (GM.Rels c) (GM.wm 1) ∈ commutator (GM.Grp c) :=
    wm_mem c hc 1
  have h_prod : PresentedGroup.mk (GM.Rels c) (GM.wm s * GM.wm 1) ∈ commutator (GM.Grp c) :=
    Subgroup.mul_mem _ h_wm_s h_wm_1
  rw [← h_eq]
  exact h_prod

theorem h1h2_inv_mem (c : ℕ → ℕ) (hc : 1 ≤ c 2) (x : ℂ×) :
  PresentedGroup.mk (GM.Rels c) (GM.h1 x * GM.h2 x-1) ∈ commutator (GM.Grp c) := by
  have hrel := GM.Rel.Re3 (c := c) (-x)
  have h_simplified : (GM.h1 (c := c) (-(-x)) * GM.h2 (c := c) (-(x-1))) = (GM.h1 (c := c) x * GM.h2 (c
↪ := c) x-1) := by
    simp
  have hmem : PresentedGroup.mk (GM.Rels c) (GM.wm (-x) * GM.wm 1) ∈ commutator (GM.Grp c) := by
    have h1 : PresentedGroup.mk (GM.Rels c) (GM.wm (-x)) ∈ commutator (GM.Grp c) := wm_mem c hc (-x)
    have h2 : PresentedGroup.mk (GM.Rels c) (GM.wm 1) ∈ commutator (GM.Grp c) := wm_mem c hc 1
    exact Subgroup.mul_mem _ h1 h2
  have heq := mk_eqr c (GM.wm (-x) * GM.wm 1) (GM.h1 x * GM.h2 x-1) (by
    simpa [h_simplified] using hrel)
  rw [← heq]
  exact hmem

theorem im3_mem (c : ℕ → ℕ) (l j k : ℕ) (h : GM.Valid c 1 j k) (s : ℂ×) :
  PresentedGroup.mk (GM.Rels c) (GM.h1 ((-s) ^ (j - 1)) * GM.h2 ((-s) ^ (1 + 1)))
  ∈ commutator (GM.Grp c) := by
  set w1 := GM.ww 1 j k h (s ^ ((1 + 1) * (j - 1))) with hw1
  set w0 := GM.ww 1 j k h 1 with hw0
  have hIm3_rel : GM.Rel c (GM.eqr (w1 * w0) (GM.h1 ((-s) ^ (j - 1)) * GM.h2 ((-s) ^ (1 + 1)))) :=
    GM.Rel.Im3 1 j k h s
  have h_mul_inv : w1 * w0 * (GM.h1 ((-s) ^ (j - 1)) * GM.h2 ((-s) ^ (1 + 1)))-1 ∈ GM.Rels c := by
    simpa [GM.Rels] using hIm3_rel
  have h_mem' : PresentedGroup.mk (GM.Rels c) (GM.eqr (w1 * w0) (GM.h1 ((-s) ^ (j - 1)) * GM.h2 ((-s) ^ (1 +
↪ 1)))) = 1 :=
    PresentedGroup.one_of_mem hIm3_rel
  have h_eq : PresentedGroup.mk (GM.Rels c) (w1 * w0) =

```

```

    PresentedGroup.mk (GM.Rels c) (GM.h1 ((-s) ^ (j - 1)) * GM.h2 ((-s) ^ (1 + 1))) := by
    apply PresentedGroup.mk_eq_mk_of_mul_inv_mem h_mul_inv
  have h_prod_mem : PresentedGroup.mk (GM.Rels c) (w1 * w0) ∈ commutator (GM.Grp c) := by
    have h1 : PresentedGroup.mk (GM.Rels c) w1 ∈ commutator (GM.Grp c) :=
      ww_mem c 1 j k h (s ^ ((1 + 1) * (j - 1)))
    have h2 : PresentedGroup.mk (GM.Rels c) w0 ∈ commutator (GM.Grp c) :=
      ww_mem c 1 j k h 1
    exact Subgroup.mul_mem (commutator (GM.Grp c)) h1 h2
  rw [← h_eq]
  exact h_prod_mem

theorem h1sq_h2_mem (c : ℕ → ℕ) (hc : 1 ≤ c 2) (y : ℂ×) :
  PresentedGroup.mk (GM.Rels c) (GM.h1 (y ^ 2) * GM.h2 y) ∈ commutator (GM.Grp c) := by
  have hvalid : GM.Valid c 0 2 1 := ⟨by decide, by decide, hc⟩
  have hw1 := ww_mem c 0 2 1 hvalid (y ^ 2)
  have hw2 := ww_mem c 0 2 1 hvalid 1
  have hprod : PresentedGroup.mk (GM.Rels c)
    (GM.ww 0 2 1 hvalid (y ^ 2) * GM.ww 0 2 1 hvalid 1) ∈ commutator (GM.Grp c) :=
    (commutator (GM.Grp c)).mul_mem hw1 hw2
  have him3 : GM.Rel c (GM.eqr (GM.ww 0 2 1 hvalid (y ^ 2) * GM.ww 0 2 1 hvalid 1)
    (GM.h1 (y ^ 2) * GM.h2 y)) := by
    have := GM.Rel.Im3 0 2 1 hvalid (-y)
    simpa [show ((-y : ℂ×) : ℂ) ^ ((0+1)*(2-0)) = (y : ℂ)^2 by norm_num] using this
  have heq : PresentedGroup.mk (GM.Rels c)
    (GM.ww 0 2 1 hvalid (y ^ 2) * GM.ww 0 2 1 hvalid 1) =
    PresentedGroup.mk (GM.Rels c) (GM.h1 (y ^ 2) * GM.h2 y) := by
    simpa [GM.Rels] using PresentedGroup.mk_eq_mk_of_mul_inv_mem him3
  rw [← heq]
  exact hprod

theorem h1_cube_eq (c : ℕ → ℕ) (z : ℂ×) :
  PresentedGroup.mk (GM.Rels c) (GM.h1 (z ^ 3)) =
  PresentedGroup.mk (GM.Rels c) (GM.h1 z * GM.h2 z-1) *
  (PresentedGroup.mk (GM.Rels c) (GM.h1 ((z-1) ^ 2) * GM.h2 z-1))-1 := by
  let φ := PresentedGroup.mk (GM.Rels c)
  have hφ_mul : ∀ x y : GM.F c, φ (x * y) = φ x * φ y := by
    intro x y; exact MonoidHom.map_mul φ x y
  have hφ_inv : ∀ x : GM.F c, φ (x-1) = (φ x)-1 := by
    intro x; exact MonoidHom.map_inv φ x

  have h1_mul : ∀ s t : ℂ×, φ (GM.h1 (c := c) (s * t)) = φ (GM.h1 (c := c) s) * φ (GM.h1 (c := c) t) := by
    intro s t
    have hRel : GM.Rel c (GM.eqr (GM.h1 (c := c) s * GM.h1 (c := c) t) (GM.h1 (c := c) (s * t))) :=
      GM.Rel.H1a (c := c) s t
    have h_one := PresentedGroup.one_of_mem (hx := hRel) (rels := GM.Rels c)

  have hcalc : φ (GM.eqr (GM.h1 (c := c) s * GM.h1 (c := c) t) (GM.h1 (c := c) (s * t))) =
    φ (GM.h1 (c := c) s * GM.h1 (c := c) t) * (φ (GM.h1 (c := c) (s * t)))-1 := by
    simp [GM.eqr, hφ_mul, hφ_inv]
  rw [hcalc] at h_one

  have h_eq : φ (GM.h1 (c := c) s * GM.h1 (c := c) t) = φ (GM.h1 (c := c) (s * t)) := by

```

```

calc
   $\varphi (GM.h1 (c := c) s * GM.h1 (c := c) t) =$ 
     $(\varphi (GM.h1 (c := c) s * GM.h1 (c := c) t) * (\varphi (GM.h1 (c := c) (s * t)))^{-1}) * \varphi (GM.h1 (c := c) (s * t)) := \text{by group}$ 
   $\_ = 1 * \varphi (GM.h1 (c := c) (s * t)) := \text{by rw [h\_one]}$ 
   $\_ = \varphi (GM.h1 (c := c) (s * t)) := \text{by simp}$ 
  rw [h $\varphi$ _mul] at h_eq
  exact h_eq.symm

have h1_one :  $\varphi (GM.h1 (c := c) 1) = 1 := \text{by}$ 
  have hRel : GM.Rel c (GM.eqr (GM.h1 (c := c) 1 * GM.h1 (c := c) 1) (GM.h1 (c := c) (1 * 1))) :=
    GM.Rel.H1a (c := c) 1 1
  have h_one := PresentedGroup.one_of_mem (hx := hRel) (rels := GM.Rels c)

have hcalc :  $\varphi (GM.eqr (GM.h1 (c := c) 1 * GM.h1 (c := c) 1) (GM.h1 (c := c) (1 * 1))) =$ 
   $\varphi (GM.h1 (c := c) 1 * GM.h1 (c := c) 1) * (\varphi (GM.h1 (c := c) (1 * 1)))^{-1} := \text{by}$ 
  simp [GM.eqr, h $\varphi$ _mul]
  rw [hcalc] at h_one

  rw [mul_one] at h_one

  rw [h $\varphi$ _mul] at h_one

calc
   $\varphi (GM.h1 (c := c) 1) = (\varphi (GM.h1 (c := c) 1) * \varphi (GM.h1 (c := c) 1)) * (\varphi (GM.h1 (c := c) 1))^{-1} := \text{by}$ 
 $\hookrightarrow \text{group}$ 
   $\_ = 1 := \text{h\_one}$ 

have h1_inv :  $\forall s : \mathbb{C}^\times, \varphi (GM.h1 (c := c) s^{-1}) = (\varphi (GM.h1 (c := c) s))^{-1} := \text{by}$ 
  intro s
  have hRel : GM.Rel c (GM.eqr (GM.h1 (c := c) s * GM.h1 (c := c) s-1) (GM.h1 (c := c) (s * s-1))) :=
    GM.Rel.H1a (c := c) s s-1

  have h_one := PresentedGroup.one_of_mem (hx := hRel) (rels := GM.Rels c)
  have hcalc :  $\varphi (GM.eqr (GM.h1 (c := c) s * GM.h1 (c := c) s^{-1}) (GM.h1 (c := c) (s * s^{-1}))) =$ 
     $\varphi (GM.h1 (c := c) s * GM.h1 (c := c) s^{-1}) * (\varphi (GM.h1 (c := c) (s * s^{-1})))^{-1} := \text{by}$ 
    simp [GM.eqr, h $\varphi$ _mul, h $\varphi$ _inv]
  rw [hcalc] at h_one

have h_s_one :  $\varphi (GM.h1 (c := c) (s * s^{-1})) = 1 := \text{by}$ 
  have :  $s * s^{-1} = (1 : \mathbb{C}^\times) := \text{by simp}$ 
  rw [this]
  exact h1_one
  rw [h_s_one] at h_one

have :  $(1 : \text{PresentedGroup } (GM.Rels c))^{-1} = 1 := \text{by simp}$ 
  rw [this, mul_one] at h_one

calc
   $\varphi (GM.h1 (c := c) s^{-1}) = 1 * \varphi (GM.h1 (c := c) s^{-1}) := \text{by simp}$ 

```

```

_ = ((φ (GM.h1 (c := c) s))-1 * φ (GM.h1 (c := c) s)) * φ (GM.h1 (c := c) s-1) := by simp
_ = (φ (GM.h1 (c := c) s))-1 * (φ (GM.h1 (c := c) s) * φ (GM.h1 (c := c) s-1)) := by group
_ = (φ (GM.h1 (c := c) s))-1 * 1 := by
  rw [show φ (GM.h1 (c := c) s) * φ (GM.h1 (c := c) s-1) = 1 from h_one]
_ = (φ (GM.h1 (c := c) s))-1 := by simp

have h_pow_h1 : ∀ (x : ℂ×) (n : ℕ), φ (GM.h1 (c := c) (x ^ n)) = (φ (GM.h1 (c := c) x)) ^ n := by
  intro x n
  induction' n with k ih
  .
  simp [h1_one]
  .
  rw [pow_succ, pow_succ]

  rw [h1_mul (x ^ k) x]
  rw [ih]

have h_inv_h1 : ∀ (x : ℂ×), φ (GM.h1 (c := c) x-1) = (φ (GM.h1 (c := c) x))-1 := h1_inv

calc
  φ (GM.h1 (c := c) (z ^ 3)) = (φ (GM.h1 (c := c) z)) ^ 3 := by rw [h_pow_h1]
  _ = (φ (GM.h1 (c := c) z)) * ((φ (GM.h1 (c := c) z))-1 ^ 2)-1 := by group
  _ = (φ (GM.h1 (c := c) z) * φ (GM.h2 (c := c) z-1)) * (((φ (GM.h1 (c := c) z))-1 ^ 2) * φ (GM.h2 (c :=
→ c) z-1))-1 := by group
  _ = (φ (GM.h1 (c := c) z) * φ (GM.h2 (c := c) z-1)) * ((φ (GM.h1 (c := c) (z-1 ^ 2)) * φ (GM.h2 (c :=
→ c) z-1))-1) := by
    simp [h_pow_h1, h_inv_h1]
  _ = PresentedGroup.mk (GM.Rels c) (GM.h1 z * GM.h2 z-1) *
    (PresentedGroup.mk (GM.Rels c) (GM.h1 ((z-1 ^ 2) * GM.h2 z-1))-1 := by
    simp [φ, hφ_mul]

theorem exists_unit_cube (s : ℂ×) : ∃ z : ℂ×, z ^ 3 = s := by
  obtain ⟨w, hw⟩ := IsAlgClosed.exists_pow_nat_eq (s : ℂ) (by decide : 0 < 3)
  have hw_ne_zero : w ≠ 0 := by
    intro hzero
    rw [hzero, zero_pow (by decide : 3 ≠ 0)] at hw
  have hs_ne_zero : (s : ℂ) ≠ 0 := Units.ne_zero s
  exact hs_ne_zero (Eq.symm hw)
  let z : ℂ× := Units.mk0 w hw_ne_zero
  refine ⟨z, ?_⟩
  apply Units.ext_iff.mpr
  calc
    (z : ℂ) ^ 3 = w ^ 3 := rfl
    _ = (s : ℂ) := hw

theorem h1_mem (c : ℕ → ℕ) (hc : 1 ≤ c 2) (s : ℂ×) :
  PresentedGroup.mk (GM.Rels c) (GM.h1 s) ∈ commutator (GM.Grp c) := by
  rcases exists_unit_cube s with ⟨z, hz⟩

  have h_comm_z : PresentedGroup.mk (GM.Rels c) (GM.h1 z * GM.h2 (z-1)) ∈ commutator (GM.Grp c) :=
    h1h2_inv_mem c hc z

```

```

have h_comm_z_inv : PresentedGroup.mk (GM.Rels c) (GM.h1 ((z-1) ^ 2) * GM.h2 (z-1)) ∈
  commutator (GM.Grp c) :=
  h1sq_h2_mem c hc (z-1)

have h_cube := h1_cube_eq c z

have h_prod : PresentedGroup.mk (GM.Rels c) (GM.h1 z * GM.h2 (z-1)) *
  (PresentedGroup.mk (GM.Rels c) (GM.h1 ((z-1) ^ 2) * GM.h2 (z-1)))-1 ∈
  commutator (GM.Grp c) := by
  apply Subgroup.mul_mem
  · exact h_comm_z
  · apply Subgroup.inv_mem
    exact h_comm_z_inv

have h_cube_mem : PresentedGroup.mk (GM.Rels c) (GM.h1 (z ^ 3)) ∈ commutator (GM.Grp c) := by
  rw [h_cube]
  exact h_prod

simp [hz] using h_cube_mem

theorem h2_eq_conj (c : ℕ → ℕ) (s : ℂ×) :
  PresentedGroup.mk (GM.Rels c) (GM.h2 s) =
    PresentedGroup.mk (GM.Rels c) (GM.wm 1) * PresentedGroup.mk (GM.Rels c) (GM.h1 s) *
      (PresentedGroup.mk (GM.Rels c) (GM.wm 1))-1 := by

have h_eq := mk_eqr c (GM.wm 1 * GM.h1 s * (GM.wm 1)-1) (GM.h2 s) (by
  apply GM.Rel.Re5a)

simp [PresentedGroup.mk, map_mul, map_inv] using h_eq.symm

theorem h2_mem (c : ℕ → ℕ) (hc : 1 ≤ c 2) (s : ℂ×) :
  PresentedGroup.mk (GM.Rels c) (GM.h2 s) ∈ commutator (GM.Grp c) := by
have hnormal : (commutator (GM.Grp c)).Normal := inferInstance
have hh1 : PresentedGroup.mk (GM.Rels c) (GM.h1 s) ∈ commutator (GM.Grp c) := h1_mem c hc s
have hww : PresentedGroup.mk (GM.Rels c) (GM.wm 1) ∈ commutator (GM.Grp c) := wm_mem c hc 1
rw [h2_eq_conj c s]
exact hnormal.conj_mem (PresentedGroup.mk (GM.Rels c) (GM.h1 s)) hh1
  (PresentedGroup.mk (GM.Rels c) (GM.wm 1))

theorem gsimple_g_m_perfect (c : ℕ → ℕ) (hc : 1 ≤ c 2) :
  commutator (GM.Grp c) = 1 := by
  apply subgroup_top_of_gens
  intro g
  cases g with
  | H1 s => exact h1_mem c hc s
  | H2 s => exact h2_mem c hc s
  | Xm u => exact xm_mem c hc u
  | Ym u => exact ym_mem c hc u
  | X l j k h u => exact xx_mem c l j k h u
  | Y l j k h u => exact yy_mem c l j k h u

-- From run srv_20261006_225233_GSimple_Theorem_3_4 (GSimple_Theorem_3_4), proved.

namespace GM

```

```

/-- Generators of the toral subgroup: the images of  $H_1(s)$  and  $H_2(s)$  in  $G(m)$ . -/
def torusGens (c :  $\mathbb{N} \rightarrow \mathbb{N}$ ) : Set (Grp c) :=
  {g |  $\exists s : \mathbb{C}^\times$ , g = PresentedGroup.mk (Rels c) (h1 s)  $\vee$ 
    g = PresentedGroup.mk (Rels c) (h2 s)}

/-- The toral subgroup H of  $G(m)$ . -/
def torusSub (c :  $\mathbb{N} \rightarrow \mathbb{N}$ ) : Subgroup (Grp c) := Subgroup.closure (torusGens c)

/-- The unipotent generators  $X_{\{-1\}}(u)$ ,  $Y_{\{-1\}}(u)$ ,  $X_{\{\ell,jk\}}(u)$ ,  $Y_{\{\ell,jk\}}(u)$ , viewed in  $G(m)$ . -/
def unipGens (c :  $\mathbb{N} \rightarrow \mathbb{N}$ ) : Set (Grp c) :=
  {g | ( $\exists u : \mathbb{C}$ , g = PresentedGroup.mk (Rels c) (xm u))  $\vee$ 
    ( $\exists u : \mathbb{C}$ , g = PresentedGroup.mk (Rels c) (ym u))  $\vee$ 
    ( $\exists (l j k : \mathbb{N})$  (h : Valid c l j k) (u :  $\mathbb{C}$ ),
      g = PresentedGroup.mk (Rels c) (xx l j k h u))  $\vee$ 
    ( $\exists (l j k : \mathbb{N})$  (h : Valid c l j k) (u :  $\mathbb{C}$ ),
      g = PresentedGroup.mk (Rels c) (yy l j k h u))}

/-- The unipotent subgroup U of  $G(m)$ . -/
def unipSub (c :  $\mathbb{N} \rightarrow \mathbb{N}$ ) : Subgroup (Grp c) := Subgroup.closure (unipGens c)

end GM

theorem h1_conj_unip (c :  $\mathbb{N} \rightarrow \mathbb{N}$ ) (s :  $\mathbb{C}^\times$ ) (g : GM.Grp c) (hg : g  $\in$  GM.unipGens c) :
  PresentedGroup.mk (GM.Rels c) (GM.h1 s) * g * (PresentedGroup.mk (GM.Rels c) (GM.h1 s))-1
   $\in$  GM.unipGens c := by

rcases hg with (<u, hg_u> | <u, hg_u> | <l, j, k, h, u, hg_u> | <l, j, k, h, u, hg_u>)
.
  subst hg_u

  have hrel := GM.Rel.Re6a (c := c) s u
  have h_eq : PresentedGroup.mk (GM.Rels c) (GM.h1 s * GM.xm u * (GM.h1 s)-1) =
    PresentedGroup.mk (GM.Rels c) (GM.xm ((s :  $\mathbb{C}$ ) * u)) := by
    simpa [GM.eqr] using mk_eqr c (GM.h1 s * GM.xm u * (GM.h1 s)-1) (GM.xm ((s :  $\mathbb{C}$ ) * u)) hrel
  have h_target : PresentedGroup.mk (GM.Rels c) (GM.xm ((s :  $\mathbb{C}$ ) * u))  $\in$  GM.unipGens c := by
    refine Or.inl ((s :  $\mathbb{C}$ ) * u, ?_)
    rfl
  have h_eq' : PresentedGroup.mk (GM.Rels c) (GM.h1 s) * PresentedGroup.mk (GM.Rels c) (GM.xm u) *
    (PresentedGroup.mk (GM.Rels c) (GM.h1 s))-1 = PresentedGroup.mk (GM.Rels c) (GM.xm ((s :  $\mathbb{C}$ ) * u)) := by
    simpa [GM.xm, map_mul, map_inv] using h_eq
  rw [h_eq']
  exact h_target
.

  subst hg_u

  have hrel := GM.Rel.Re6c (c := c) s u
  have h_eq : PresentedGroup.mk (GM.Rels c) (GM.h1 s * GM.ym u * (GM.h1 s)-1) =
    PresentedGroup.mk (GM.Rels c) (GM.ym (((s :  $\mathbb{C}$ )-1) * u)) := by
    simpa [GM.eqr] using mk_eqr c (GM.h1 s * GM.ym u * (GM.h1 s)-1) (GM.ym (((s :  $\mathbb{C}$ )-1) * u)) hrel
  have h_target : PresentedGroup.mk (GM.Rels c) (GM.ym (((s :  $\mathbb{C}$ )-1) * u))  $\in$  GM.unipGens c := by
    refine Or.inr (Or.inl (((s :  $\mathbb{C}$ )-1) * u, ?_))
    rfl
  have h_eq' : PresentedGroup.mk (GM.Rels c) (GM.h1 s) * PresentedGroup.mk (GM.Rels c) (GM.ym u) *
    (PresentedGroup.mk (GM.Rels c) (GM.h1 s))-1 = PresentedGroup.mk (GM.Rels c) (GM.ym (((s :  $\mathbb{C}$ )-1) * u))
   $\Rightarrow$  := by
    simpa [GM.ym, map_mul, map_inv] using h_eq

```

```

rw [h_eq']
exact h_target
.

subst hg_u

have hrel := GM.Rel.Im6a (c := c) l j k h s u
have h_eq : PresentedGroup.mk (GM.Rels c) (GM.h1 s * GM.xx l j k h u * (GM.h1 s)-1) =
  PresentedGroup.mk (GM.Rels c) (GM.xx l j k h ((s : ℂ) ^ (1 + 1) * u)) := by
  simpa [GM.eqr] using mk_eqr c (GM.h1 s * GM.xx l j k h u * (GM.h1 s)-1) (GM.xx l j k h ((s : ℂ) ^ (1 +
→ 1) * u)) hrel
have h_target : PresentedGroup.mk (GM.Rels c) (GM.xx l j k h ((s : ℂ) ^ (1 + 1) * u)) ∈ GM.unipGens c
→ := by
  refine Or.inr (Or.inr (Or.inl ⟨l, j, k, h, ((s : ℂ) ^ (1 + 1) * u), ?_⟩))
  rfl
have h_eq' : PresentedGroup.mk (GM.Rels c) (GM.h1 s) * PresentedGroup.mk (GM.Rels c) (GM.xx l j k h u) *
  (PresentedGroup.mk (GM.Rels c) (GM.h1 s))-1 = PresentedGroup.mk (GM.Rels c) (GM.xx l j k h ((s : ℂ) ^
→ (1 + 1) * u)) := by
  simpa [GM.xx, map_mul, map_inv] using h_eq
rw [h_eq']
exact h_target
.

subst hg_u

have hrel := GM.Rel.Im6c (c := c) l j k h s u
have h_eq : PresentedGroup.mk (GM.Rels c) (GM.h1 s * GM.yy l j k h u * (GM.h1 s)-1) =
  PresentedGroup.mk (GM.Rels c) (GM.yy l j k h (((s : ℂ) ^ (1 + 1))-1 * u)) := by
  simpa [GM.eqr] using mk_eqr c (GM.h1 s * GM.yy l j k h u * (GM.h1 s)-1) (GM.yy l j k h (((s : ℂ) ^ (1
→ + 1))-1 * u)) hrel
have h_target : PresentedGroup.mk (GM.Rels c) (GM.yy l j k h (((s : ℂ) ^ (1 + 1))-1 * u)) ∈ GM.unipGens
→ c := by
  refine Or.inr (Or.inr (Or.inr ⟨l, j, k, h, (((s : ℂ) ^ (1 + 1))-1 * u), ?_⟩))
  rfl
have h_eq' : PresentedGroup.mk (GM.Rels c) (GM.h1 s) * PresentedGroup.mk (GM.Rels c) (GM.yy l j k h u) *
  (PresentedGroup.mk (GM.Rels c) (GM.h1 s))-1 = PresentedGroup.mk (GM.Rels c) (GM.yy l j k h (((s : ℂ)
→ ^ (1 + 1))-1 * u)) := by
  simpa [GM.yy, map_mul, map_inv] using h_eq
rw [h_eq']
exact h_target

theorem h2_conj_unip (c : ℕ → ℕ) (s : ℂ×) (g : GM.Grp c) (hg : g ∈ GM.unipGens c) :
  PresentedGroup.mk (GM.Rels c) (GM.h2 s) * g * (PresentedGroup.mk (GM.Rels c) (GM.h2 s))-1
  ∈ GM.unipGens c := by
  rcases hg with ⟨u, hg'⟩ | ⟨u, hg'⟩ | ⟨l, j, k, h, u, hg'⟩ | ⟨l, j, k, h, u, hg'⟩
  .
  subst hg'
  have h_rel := GM.Rel.Re6b (c := c) s u
  have h_eq := mk_eqr c (GM.h2 s * GM.xm u * (GM.h2 s)-1) (GM.xm ((s : ℂ)-1 * u)) h_rel
  have h_eq' : PresentedGroup.mk (GM.Rels c) (GM.h2 s) * PresentedGroup.mk (GM.Rels c) (GM.xm u) *
    (PresentedGroup.mk (GM.Rels c) (GM.h2 s))-1 = PresentedGroup.mk (GM.Rels c) (GM.xm ((s : ℂ)-1 * u))
  → := by
    simpa [PresentedGroup.mk, map_mul, map_inv] using h_eq
  rw [h_eq']
  exact Set.mem_setOf.mpr (Or.inl ⟨(s : ℂ)-1 * u, rfl⟩)
  .
  subst hg'
  have h_rel := GM.Rel.Re6d (c := c) s u
  have h_eq := mk_eqr c (GM.h2 s * GM.ym u * (GM.h2 s)-1) (GM.ym ((s : ℂ) * u)) h_rel
  have h_eq' : PresentedGroup.mk (GM.Rels c) (GM.h2 s) * PresentedGroup.mk (GM.Rels c) (GM.ym u) *
    (PresentedGroup.mk (GM.Rels c) (GM.h2 s))-1 = PresentedGroup.mk (GM.Rels c) (GM.ym ((s : ℂ) * u)) := by

```

```

    simpa [PresentedGroup.mk, map_mul, map_inv] using h_eq
    rw [h_eq']
    exact Set.mem_setOf.mpr (Or.inr (Or.inl ⟨(s : ℂ) * u, rfl⟩))
  .
  subst hg'
  have h_rel := GM.Rel.Im6b (c := c) 1 j k h s u
  have h_eq := mk_eqr c (GM.h2 s * GM.xx 1 j k h u * (GM.h2 s)-1) (GM.xx 1 j k h (((s : ℂ) ^ (j - 1)) *
→ u)) h_rel
  have h_eq' : PresentedGroup.mk (GM.Rels c) (GM.h2 s) * PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h u) *
    (PresentedGroup.mk (GM.Rels c) (GM.h2 s))-1 = PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h (((s : ℂ)
→ ^ (j - 1)) * u)) := by
    simp [PresentedGroup.mk, map_mul, map_inv] using h_eq
    rw [h_eq']
    exact Set.mem_setOf.mpr (Or.inr (Or.inr (Or.inl ⟨1, j, k, h, ((s : ℂ) ^ (j - 1)) * u, rfl⟩)))
  .
  subst hg'
  have h_rel := GM.Rel.Im6d (c := c) 1 j k h s u
  have h_eq := mk_eqr c (GM.h2 s * GM.yy 1 j k h u * (GM.h2 s)-1) (GM.yy 1 j k h (((s : ℂ) ^ (j - 1))-1
→ * u)) h_rel
  have h_eq' : PresentedGroup.mk (GM.Rels c) (GM.h2 s) * PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h u) *
    (PresentedGroup.mk (GM.Rels c) (GM.h2 s))-1 = PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h (((s : ℂ)
→ ^ (j - 1))-1 * u)) := by
    simp [PresentedGroup.mk, map_mul, map_inv] using h_eq
    rw [h_eq']
    exact Set.mem_setOf.mpr (Or.inr (Or.inr (Or.inr ⟨1, j, k, h, (((s : ℂ) ^ (j - 1))-1 * u, rfl⟩)))
  .
theorem h1_inv_mk (c : ℕ → ℕ) (s : ℂ×) :
  PresentedGroup.mk (GM.Rels c) (GM.h1 s-1) = (PresentedGroup.mk (GM.Rels c) (GM.h1 s))-1 := by
  let φ := PresentedGroup.mk (GM.Rels c)
  have hφ_mul : ∀ x y : GM.F c, φ (x * y) = φ x * φ y := by
    intro x y; exact MonoidHom.map_mul φ x y
  have hφ_inv : ∀ x : GM.F c, φ (x-1) = (φ x)-1 := by
    intro x; exact MonoidHom.map_inv φ x
  have h1_one : φ (GM.h1 1) = 1 := by
    have hRel : GM.Rel c (GM.eqr (GM.h1 1 * GM.h1 1) (GM.h1 (1 * 1))) :=
      GM.Rel.H1a (c := c) 1 1
    have h_one := PresentedGroup.one_of_mem (hx := hRel) (rels := GM.Rels c)
    have hcalc : φ (GM.eqr (GM.h1 1 * GM.h1 1) (GM.h1 (1 * 1))) =
      φ (GM.h1 1 * GM.h1 1) * (φ (GM.h1 (1 * 1)))-1 := by
      simp [GM.eqr, hφ_mul, hφ_inv]
    rw [hcalc] at h_one
    rw [mul_one] at h_one
    rw [hφ_mul] at h_one
    calc
      φ (GM.h1 1) = (φ (GM.h1 1) * φ (GM.h1 1)) * (φ (GM.h1 1))-1 := by group
      _ = 1 := h_one
  have hRel : GM.Rel c (GM.eqr (GM.h1 s * GM.h1 s-1) (GM.h1 (s * s-1))) :=
    GM.Rel.H1a (c := c) s s-1
  have hRel' : GM.Rel c (GM.eqr (GM.h1 s * GM.h1 s-1) (GM.h1 1)) := by
    simp using hRel
  have h_eq := mk_eqr c (GM.h1 s * GM.h1 s-1) (GM.h1 1) hRel'
  calc
    φ (GM.h1 s-1) = 1 * φ (GM.h1 s-1) := by simp
    _ = ((φ (GM.h1 s))-1 * φ (GM.h1 s)) * φ (GM.h1 s-1) := by simp
    _ = (φ (GM.h1 s))-1 * (φ (GM.h1 s) * φ (GM.h1 s-1)) := by group
    _ = (φ (GM.h1 s))-1 * φ (GM.h1 s * GM.h1 s-1) := by rw [hφ_mul]
    _ = (φ (GM.h1 s))-1 * φ (GM.h1 1) := by rw [h_eq]
    _ = (φ (GM.h1 s))-1 * 1 := by rw [h1_one]
    _ = (φ (GM.h1 s))-1 := by simp

```

```

theorem h2_inv_mk (c : ℕ → ℕ) (s : ℂ×) :
  PresentedGroup.mk (GM.Rels c) (GM.h2 s-1) = (PresentedGroup.mk (GM.Rels c) (GM.h2 s))-1 := by
  rw [h2_eq_conj c s-1, h2_eq_conj c s]
  simp
  rw [h1_inv_mk c s]
  group

theorem torusGens_conj (c : ℕ → ℕ) :
  ∀ t ∈ GM.torusGens c, ∀ g ∈ GM.unipGens c, t * g * t-1 ∈ GM.unipGens c := by
  intro t ht g hg
  rcases ht with ⟨s, hs⟩
  rcases hs with (hs | hs)
  ·
    rw [hs]
    exact h1_conj_unip c s g hg
  ·
    rw [hs]
    exact h2_conj_unip c s g hg

theorem torusGens_inv (c : ℕ → ℕ) :
  ∀ t ∈ GM.torusGens c, t-1 ∈ GM.torusGens c := by
  intro t ht
  rcases ht with ⟨s, hs | hs⟩
  · rw [hs]
    refine ⟨s-1, Or.inl ?_⟩
    rw [h1_inv_mk c s]
  · rw [hs]
    refine ⟨s-1, Or.inr ?_⟩
    rw [h2_inv_mk c s]

theorem conj_closure_of_gens {G : Type*} [Group G] (T : Set G) (x : G)
  (hx : ∀ g ∈ T, x * g * x-1 ∈ Subgroup.closure T) :
  ∀ u ∈ Subgroup.closure T, x * u * x-1 ∈ Subgroup.closure T := by
  intro u hu
  induction hu using Subgroup.closure_induction with
  | mem g hg =>
    exact hx g hg
  | one =>
    simp
  | mul a b ha hb iha ihb =>
    have h : x * (a * b) * x-1 = (x * a * x-1) * (x * b * x-1) := by group
    rw [h]
    exact Subgroup.mul_mem _ iha ihb
  | inv a ha iha =>
    have hxa : x * a-1 * x-1 = (x * a * x-1)-1 := by group
    rw [hxa]
    exact Subgroup.inv_mem _ iha

theorem closure_normalizes {G : Type*} [Group G] (S T : Set G)
  (hinv : ∀ t ∈ S, t-1 ∈ S)
  (hconj : ∀ t ∈ S, ∀ u ∈ Subgroup.closure T, t * u * t-1 ∈ Subgroup.closure T) :
  ∀ h ∈ Subgroup.closure S, ∀ u ∈ Subgroup.closure T, h * u * h-1 ∈ Subgroup.closure T := by
  intro h hh
  induction hh using Subgroup.closure_induction_left with
  | one =>
    intro u hu
    simp using hu
  | mul_left x hx y hy ih =>

```

```

intro u hu
have hcalc : (x * y) * u * (x * y)-1 = x * (y * u * y-1) * x-1 := by
  simp [mul_assoc]
rw [hcalc]
exact hconj x hx (y * u * y-1) (ih u hu)
| inv_mul_cancel x hx y hy ih =>
  intro u hu
  have hcalc : (x-1 * y) * u * (x-1 * y)-1 = x-1 * (y * u * y-1) * x := by
    simp [mul_assoc]
  rw [hcalc]
  have hcalc2 : x-1 * (y * u * y-1) * x = x-1 * (y * u * y-1) * (x-1)-1 := by
    simp
  rw [hcalc2]
  have hx_inv : x-1 ∈ S := hinv x hx
  exact hconj x-1 hx_inv (y * u * y-1) (ih u hu)

theorem gsimple_torus_normalizes_unipotent (c : ℕ → ℕ) :
  ∀ h ∈ GM.torusSub c, ∀ u ∈ GM.unipSub c, h * u * h-1 ∈ GM.unipSub c := by
  intro h hh u hu
  exact closure_normalizes (GM.torusGens c) (GM.unipGens c) (torusGens_inv c)
  (fun t ht => conj_closure_of_gens (GM.unipGens c) t
    (fun g hg => Subgroup.subset_closure (torusGens_conj c t ht g hg))) h hh u hu

theorem conj_inv_rewrite {G : Type*} [Group G] (h u : G) :
  u * h = h * (h-1 * u * h-1-1) := by
  group

theorem solution (c : ℕ → ℕ) :
  ∀ h ∈ GM.torusSub c, ∀ u ∈ GM.unipSub c,
  ∃ u' ∈ GM.unipSub c, u * h = h * u' := by
  intro h hh u hu
  refine ⟨h-1 * u * h-1-1,
    gsimple_torus_normalizes_unipotent c h-1 (Subgroup.inv_mem _ hh) u hu, ?_⟩
  exact conj_inv_rewrite h u

```